PRODUCT & ENGINEERING DOSSIER

Atithya

Food ordering & kitchen order management

A complete guide to the customer experience, operational workflow, system architecture, API, security, testing and deployment.

LIVE APPLICATION
atithya-kitchen.vercel.app

SOURCE CODE
github.com/Pranitej/atithya-order-management

Documentation snapshot: 14 October 2026 · main branch · da34793

Contents

Part 1: Feature guide 6

1.1 What Atithya is 6

1.2 The pages at a glance 6

1.3 Accounts and signing in 6

1.4 Browsing the menu 7

1.5 The cart 8

1.6 Checkout 9

1.7 Tracking an order live 10

1.8 Cancelling an order 11

1.9 Your orders 11

1.10 The kitchen board 12

1.11 Messages you may see 12

1.12 Phones and larger screens 13

1.13 Accessibility 14

1.14 The full menu 14

Pizza (4) 14

Burgers (4) 14

Indian Mains (5) 15

South Indian (2) 15

Chinese (3) 15

Pasta (2) 15

Wraps and Sandwiches (3) 15

Salads and Bowls (2) 15

Sides (5) 16

Desserts (6) 16

Drinks (7) 16

1.15 What the demo does not do 16

Part 2: Technical documentation 17

2.1 What was asked and what was built 17

2.2 Technology stack 17

2.3 Architecture 18

Layers 19

What may import what 19

Folder structure 19

2.4 How a request is handled 20

2.5 Data model 21

users 21

sessions 22

menuitems 22

orders 22
ratelimits 23
2.6 API reference 24
Conventions 24
Endpoints at a glance 24
Error codes 24
Shapes 25
Menu 26
Orders 26
Authentication 28
Trying it from a terminal 28
2.7 Authentication and sessions 29
The two tokens 29
The cookies 29
Sessions in the database 30
Signing in 30
An ordinary request 30
Refreshing, and catching a stolen token 30
Signing out 31
In the browser 31
Who may do what 31
What follows from this design 31
2.8 Order lifecycle 31
Statuses and legal moves 31
Placing an order 32
Changing a status safely 32
2.9 The simulated kitchen 33
How overdue steps are applied 33
Where settling happens 34
The arrival estimate 34
2.10 Live updates 34
Server-Sent Events, not sockets 34
The server side 34
The browser side 35
What it costs 36
2.11 Money and pricing 36
2.12 Validation 37
2.13 The menu catalogue and seeding 38
One source for the menu 38
A dish can never be on the menu twice 38
What npm run seed does 38
Demo accounts 38
Changing the menu 38
2.14 Security 39
How the rate limit works 39

Where the protection stops 40
2.15 Front end 40
How pages are rendered 40
State 41
The cart and hydration 41
The API client 41
Components 42
Design system 42
Accessibility in the code 43
2.16 One order from start to finish 43
2.17 Testing 44
Approach 44
Three test projects 44
How the API tests work 46
How the UI tests work 46
Running the tests 47
2.18 Configuration 47
2.19 Running it locally 47
2.20 Deployment and operations 48
Hosting 48
Continuous integration 48
Routine tasks 49
2.21 Design decisions and trade-offs 49
2.22 Known limitations and next steps 50
2.23 How it was built 50
2.24 Where to find things 51
2.25 Glossary 52

Project snapshot

43	7	12	5	339
menu dishes	application pages	REST endpoints	database collections	automated tests

Atithya is the order management part of a food delivery app. A customer browses a menu, fills a cart, checks out, and then watches the order move from the kitchen to their door in real time. Kitchen staff get a board where they can see every order and move it along by hand.

It was built for the RaftLabs Senior Full Stack assessment.

- **Live app:** https://atithya-kitchen.vercel.app
- **Source code:** https://github.com/Pranitej/atithya-order-management
- **Trying it:** the sign-in page has two one-click demo accounts, "Continue as demo customer" and "Continue as kitchen staff". Nothing has to be installed and no sign-up is needed.
- **State described:** the `main` branch as of 14 October 2026 (commit `da34793`).

This document has two parts. Each can be read without the other.

- Part 1: Feature guide describes what the app does, screen by screen, for the two kinds of people who use it: customers and kitchen staff. It needs no technical background.
- Part 2: Technical documentation describes how the app is built: its architecture, data model, API, security, tests, and how to run and deploy it. It is written for a developer who is new to the codebase.

The project in numbers:

What	How many
Dishes on the menu	43, in 11 sections
Pages	7
REST endpoints	12, under `/api/v1`
Database collections	5
Automated tests	339 (94 unit, 109 API, 136 UI), all passing

Part 1: Feature guide

1.1 What Atithya is

Atithya is a food ordering website for a single kitchen. Its promise, printed across the top of the home page, is "Good food feels like home." You choose dishes from the menu, tell the kitchen where to bring them, and then follow the order on a page that updates by itself as the order is received, cooked, sent out and delivered.

Two kinds of people use it:

- **Customers** browse the menu, order, track their orders, and can cancel an order the kitchen has not started yet.
- **Kitchen staff** see every customer's orders on one board and move each order on to its next stage.

Atithya is a demonstration, and says so at the foot of every page. No real food is cooked, no rider is sent and no payment is taken. The kitchen is simulated: on the live site an order moves to its next stage by itself every 20 seconds, so the whole journey from "Order Received" to "Delivered" can be watched in about a minute.

1.2 The pages at a glance

Page	Address	Who can use it	What it is for
Menu	`/`	Anyone	Browse dishes and fill the cart
Sign in	`/login`	Anyone	Sign in, or use a demo account
Create account	`/register`	Anyone	Make a new customer account
Checkout	`/checkout`	Anyone can open it; placing the order needs an account	Give delivery details and place the order
Your orders	`/orders`	Anyone signed in	Current and past orders
Track your order	`/orders/<order id>`	The customer who placed the order, and kitchen staff	The live status of one order
Kitchen orders	`/admin/orders`	Kitchen staff	See all orders and move them to their next stage

The bar across the top of every page stays in view while you scroll. It holds:

- the Atithya logo, which always takes you back to the menu;
- **Kitchen**, shown only to kitchen staff;
- **Orders**, shown once you are signed in;
- **Sign in**, or **Sign out** once you are signed in;
- the **Cart** button, with a gold badge counting the items inside.

1.3 Accounts and signing in

You do not need an account to look at the menu or to fill a cart. You need one to place an order, because an order belongs to an account. That is what lets you track it, find it again later, and be the only customer who can see it.

Trying the app without signing up. The sign-in page has a panel headed "Just looking around? Use a ready-made account." with two buttons:

- **Continue as demo customer** signs you in as a customer called Asha Rao.
- **Continue as kitchen staff** signs you in with the kitchen's account, which can open the kitchen board.

The demo accounts are shared by everyone who tries the app, so you may see orders that other visitors placed with them.

Creating an account. The "Create your account" page asks for three things:

Field	Rule	Message if it is wrong
Your name	2 to 80 characters	"Enter your name" or "Name is too long"
Email	A valid email address	"Enter a valid email address"
Password	8 to 72 characters	"Use at least 8 characters" or "Use at most 72 characters"

Capital letters and stray spaces in an email are ignored, so `Asha@Example.com` and `asha@example.com` are the same account. If the email already has an account, the page says "An account with this email already exists". Once the account is created you are signed in straight away and returned to wherever you were heading, for example the checkout.

Signing in. Enter your email and password. A wrong password and an email that has no account both produce the same message, "Email or password is incorrect", so the page never reveals which emails are registered.

Staying signed in. You stay signed in for seven days after your last visit, including after closing the browser. Every visit within that time renews the seven days.

Signing out. "Sign out" in the top bar ends the session on the server as well as in the browser, and takes you back to the menu. Your cart is kept.

Too many attempts. Sign-in and sign-up together are limited to 10 attempts a minute from one network address. Past that, the page says "Too many requests. Please wait a moment and try again." and works again within a minute.

Kinds of account. There are two: customer and kitchen staff. Every account created through the site is a customer account. The kitchen's account is created when the database is set up, and there is no way to make a staff account from the site.

1.4 Browsing the menu

The home page opens with a green banner and, beneath it, the menu. The banner and the frame of the page appear at once. The dishes arrive a moment later, and grey placeholder blocks hold their place until they do.

Sections. The menu has 43 dishes in 11 sections, always in the same sequence: Pizza, Burgers, Indian Mains, South Indian, Chinese, Pasta, Wraps & Sandwiches, Salads & Bowls, Sides, Desserts and Drinks. Each section has a heading with its dishes beneath, in an order set by the kitchen. The complete list is in 1.14.

Each dish shows:

- a vegetarian or non-vegetarian mark;
- its name;
- its price in rupees;
- a one-line description of what is in it;
- a photo;
- an **Add** button on the lower edge of the photo.

Vegetarian and non-vegetarian marks. These are the marks printed on food packaging in India. A green dot inside a green square means vegetarian. A red-brown triangle inside a red-brown square means non-vegetarian. The shapes differ as well as the colours, so the mark can be read without relying on colour. 30 of the 43 dishes are vegetarian.

The section bar. Under the banner is a row of buttons: **All**, then one for each section. It stays pinned under the top bar as you scroll, so it is always within reach.

- Choosing a section shows only that section's dishes. Choosing **All** shows everything again. The chosen button is filled in green.
- The page moves back to the start of the menu when you choose, so you are never left looking at the foot of the page after a long list shrinks to a short one.
- The row never wraps onto a second line. When the buttons do not all fit, the row scrolls sideways: swipe it on a touch screen or a trackpad. On screens wider than a phone, a round arrow appears at whichever edge has more sections hidden beyond it, and moves the row along by about three quarters of its width.
- The button you choose slides to the middle of the row, so its neighbours on both sides stay visible.
- A section gets a button only if it has at least one dish.

If the kitchen has nothing available, the page says "The kitchen is closed right now" instead of showing an empty list.

1.5 The cart

Adding dishes. Press **Add** on a dish and it goes into the cart with a quantity of one. The Add button turns into a small control showing the quantity between a minus and a plus button, so you can change the quantity right there on the menu without opening the cart.

Opening the cart. Press the Cart button in the top bar. On a phone, a green bar also appears along the bottom of the menu as soon as the cart has something in it. It shows the number of items and what they cost, with "View cart". It is there because the top of a phone screen is a long reach for a thumb. The cart slides in from the right as a panel over the page. Close it with the X, with the Escape key, or by clicking outside it.

What the cart shows. One line for each dish: its photo, veg mark, name, the price of one, a quantity control, and the cost of that line. Under the list is the bill.

Quantities.

- Plus adds one, up to 20 of any one dish. At 20 the plus button switches off.
- Minus takes one away. Taking away the last one removes the dish from the cart.
- Adding a dish that is already in the cart raises its quantity. A dish never appears on two lines.
- A cart holds up to 30 different dishes, the most one order can have. Once it does, every other dish on the menu shows a greyed-out "Cart full" button instead of Add. Dishes already in the cart can still go up to 20.
- The badge on the Cart button counts items, not lines: two pizzas and one portion of fries make three.

The bill. The cart and the checkout both show the same four lines:

Line	How it is worked out
Subtotal	The price of each dish times its quantity, added up
Delivery	₹40, or "Free" when the subtotal is ₹500 or more
Taxes	5% of the subtotal, rounded to the nearest paisa
Total	Subtotal plus delivery plus taxes

While the subtotal is under ₹500, the cart says how much more would make delivery free, for example "Add ₹53 more for free delivery".

Two examples:

Cart	Subtotal	Delivery	Taxes	Total
1 Margherita Pizza (₹249) and 2 Masala Fries (₹99 each)	₹447	₹40	₹22.35	₹509.35
The same, plus 1 Chocolate Brownie (₹149)	₹596	Free	₹29.80	₹625.80

The cart is remembered. It is saved in your browser, so it survives reloading the page, closing the tab, and signing in or out. It belongs to the browser, not to your account: the same account on another device starts with an empty cart. It is emptied when you place an order.

An empty cart says "Your cart is empty. Add a dish from the menu and it will show up here."

Go to checkout at the foot of the cart takes you to the next step.

1.6 Checkout

The checkout page has two columns on a wide screen and one on a phone: the delivery form, and a summary headed "Your order" listing each dish with its quantity and cost, followed by the bill.

If you are not signed in, the page says "Sign in to place your order. Your cart is saved. Signing in lets you track the order live." You can sign in right there, use "Continue as demo customer", or follow "Create an account", which brings you back to the checkout afterwards. Your cart is untouched throughout.

If you are signed in, the page asks "Where should we deliver?" and shows three fields. Your name is already filled in from your account.

Field	Rule	Message if it is wrong
Full name	2 to 80 characters	"Enter your full name" or "Name is too long"
Delivery address	10 to 300 characters. The hint asks for "Flat or house number, street, area and PIN code."	"Enter your full delivery address" or "Address is too long"
Phone number	10 to 15 digits, with an optional + in front. Spaces, dashes and brackets are allowed and are removed.	"Enter a valid phone number (10 to 15 digits)"

So `+91 98765 43210`, `(987) 654-3210` and `9876543210` are all accepted. Spaces at the start and end of any field are ignored.

Placing the order. Press **Place order**.

- If anything is missing or wrong, nothing is sent. Each problem is shown in red under its field, and the cursor moves to the first field that needs fixing. A message disappears as soon as its field is corrected.
- The button shows a spinner and cannot be pressed twice while the order is being placed.
- The kitchen's system checks the same rules again on its side, so an order can never be placed with details the form would have refused.
- On success the page says "Order placed. Taking you to live tracking.", the cart is emptied, and the tracking page opens.

Payment. There is none. Under the button the page says "Pay the rider by cash or UPI when your food arrives."

What the kitchen may refuse.

- If a dish was taken off the menu after you put it in your cart, the page says, for example, "Masala Dosa is no longer available. Remove it from your cart to continue." Your cart and your details are kept.
- An order can hold at most 20 of one dish and at most 30 different dishes.

- One network address can place at most 10 orders a minute.

Prices are set by the kitchen, not by the cart. The cart shows the price each dish had when you added it. When the order is placed, the bill is worked out again from the menu's current prices, and that is the bill shown on the tracking page.

1.7 Tracking an order live

After checkout you land on the order's own page. It can be opened again at any time from **Orders** in the top bar.

What the page shows.

- The order number, for example "Order #A1B2C3". It is short enough to read out over the phone.
- A headline that says, in plain words, where the order is.
- "Arriving by" with a time, while the order is still on its way.
- A line telling you whether the page is receiving live updates.
- A timeline of the order's stages.
- A summary: what was ordered, the bill, and "Delivering to" with the name, address and phone number you gave.

The stages.

Stage	Headline on the page	What it means
Order Received	"We have your order"	The kitchen has the order and has not started yet
Preparing	"Your food is being prepared"	The kitchen is cooking
Out for Delivery	"Your order is on its way"	The rider has the food
Delivered	"Delivered. Enjoy your meal."	The order is complete
Cancelled	"This order was cancelled"	The order was stopped before it left the kitchen

The timeline lists the four stages from top to bottom, with the time each one was reached.

- A stage that is finished has a green tick.
- The stage the order is in now is larger and has a gold dot with a slow pulse, the one moving thing on the page.
- A stage still to come is grey with a hollow dot and no time.
- A cancelled order shows the stages it reached, then a red "Cancelled" row with the time. Stages it never reached are left out.

It updates by itself. You never need to reload. When the order moves on, the headline, the timeline and the arrival time change within about two seconds. The line under the headline tells you the state of the live connection:

Message	Meaning
"Connecting to live updates"	The page has just opened
"Live updates on"	Connected. What you see is current.
"Reconnecting to live updates"	The connection dropped, for example on a train. The browser is retrying by itself.
"Live updates are paused. Reload the page to reconnect."	The page gave up retrying. It shows the last status it knew.

The line disappears once the order is delivered or cancelled, because nothing more will happen to it.

How fast it moves. The kitchen is simulated. On the live site each stage lasts 20 seconds:

Time after ordering	Stage
0 seconds	Order Received
20 seconds	Preparing
40 seconds	Out for Delivery
60 seconds	Delivered

Kitchen staff can also move an order on by hand at any moment, and the countdown to the following stage then starts afresh.

The arrival time is worked out from the stages that remain. It holds steady while the order runs on schedule, moves earlier if staff move the order ahead, and disappears once the order is delivered or cancelled. All times are shown in your own time zone.

You do not have to watch. The order keeps moving if you close the page. When you come back, the timeline shows the time each stage was due, not the time you looked.

When it is over, an **Order something else** button takes you back to the menu.

1.8 Cancelling an order

A customer can cancel an order until the kitchen starts preparing it.

- While the order is "Order Received", the tracking page shows a **Cancel order** button with the note "You can cancel until the kitchen starts preparing your food." On the live site that is the first 20 seconds, unless staff start it sooner.
- Once the order is "Preparing", the button is gone.
- If the kitchen starts in the instant before you press the button, the order is not cancelled and the page says, for example, `This order is already "Preparing" and can no longer be cancelled.`
- A cancelled order is never erased. It stays in your order list, marked "Cancelled", with its timeline.

Kitchen staff can cancel a little later than a customer can: while an order is "Order Received" or "Preparing". Nobody can cancel an order that is out for delivery or delivered.

1.9 Your orders

Orders in the top bar opens the list of your orders, newest first. It shows your 50 most recent.

Each row shows the order number, a coloured status label, what was ordered (for example "2 × Masala Fries, 1 × Margherita Pizza"), when it was placed, and the total. Pressing a row opens that order's tracking page.

- While any order in the list is still in progress, the list refreshes itself every five seconds, so the status labels keep up.
- If you are not signed in, the page says "Sign in to see your orders".
- If you have never ordered, it says "You have not ordered anything yet" and offers "Browse the menu".

Your orders are private. A customer sees only their own orders. If someone opens the link to an order that is not theirs, the page says "We could not find that order", exactly as it would for an order that does not exist.

Signed in with the kitchen's account, this page lists every customer's orders, because staff are allowed to see them all. The kitchen board is the better view for that.

1.10 The kitchen board

Kitchen staff see a **Kitchen** link in the top bar. It opens the board at `/admin/orders`.

What it shows. Every customer's orders, newest first (the 50 most recent). Each row has:

- the order number and its status label;
- the customer's name and delivery address;
- what was ordered;
- the total and when the order was placed;
- the buttons for what can be done next.

Moving an order on. Each order has one button for its next stage:

The order is	Button	The order becomes
Order Received	Start preparing	Preparing
Preparing	Send out for delivery	Out for Delivery
Out for Delivery	Mark as delivered	Delivered

A red **Cancel order** button is also shown while the order is "Order Received" or "Preparing".

The rules staff work within:

- An order moves one stage at a time. It cannot skip a stage or go backwards.
- Nothing can be done to an order that is delivered or cancelled, so those rows have no buttons.
- The customer sees a change on their tracking page within about two seconds.
- While a change is being saved, the button shows a spinner and the row's other button is switched off.

The board keeps itself up to date. It refreshes every five seconds, so new orders appear without a reload. Orders also move on by themselves, because the kitchen is simulated. Moving an order by hand restarts its countdown to the following stage.

If two things change an order at once, for example two members of staff, or a member of staff and the simulation, only one change is applied. The other person sees either "This order was just updated. Refresh and try again." or a message naming the move that is no longer possible.

Who can open it. A signed-in customer who opens the address sees "This page is for kitchen staff". A visitor sees "Sign in as kitchen staff". In neither case is any order loaded.

1.11 Messages you may see

Every empty screen and every failure says what happened and offers a next step.

Where	Message	What to do
Menu	"The kitchen is closed right now"	Nothing is available. Check back later.
Cart	"Your cart is empty"	Add a dish from the menu.
Menu	"Cart full", on a dish's button	The cart already holds 30 different dishes. Remove one to add another.
Checkout	"Your cart is empty"	Press "Browse the menu".
Checkout	"Sign in to place your order"	Sign in, use the demo customer, or create an account.

Where	Message	What to do
Checkout	" is no longer available. Remove it from your cart to continue."	Remove that dish in the cart and place the order again.
Sign in	"Email or password is incorrect"	Check both and try again.
Create account	"An account with this email already exists"	Sign in instead.
Any form	"Too many requests. Please wait a moment and try again."	Wait up to a minute.
Anywhere	"Could not reach the server. Check your connection and try again."	Check your internet connection.
Tracking	"Sign in to see this order"	Your session ended. Sign in and you are brought back.
Tracking	"We could not find that order"	The link is wrong, or the order is another account's.
Tracking	"This order could not be loaded"	Press "Try again".
Tracking	"Live updates are paused. Reload the page to reconnect."	Reload the page.
Tracking	`This order is already "Preparing" and can no longer be cancelled`	The kitchen has started. The order goes ahead.
Your orders	"Sign in to see your orders"	Sign in.
Your orders	"You have not ordered anything yet"	Press "Browse the menu".
Kitchen board	"Sign in as kitchen staff"	Sign in with the kitchen's account.
Kitchen board	"This page is for kitchen staff"	Customer accounts cannot manage orders.
Kitchen board	"No orders yet"	Orders appear here as customers place them.
Kitchen board	"This order was just updated. Refresh and try again."	Someone else changed the order first.
Any address	"That page is not on the menu"	The address is wrong. Press "Back to the menu".
Any page	"This page could not be loaded. Something went wrong on our side. Your cart is safe."	Press "Try again".

1.12 Phones and larger screens

The app is laid out for a phone first and opens up as the screen gets wider.

Part of the app	On a phone	On a wider screen
Top bar	The emblem alone, and icons for Kitchen, Orders, Sign out and Cart	The full logo, with a word beside each icon
Banner	Headline and one sentence	The same, with a photo of a plate half off the right edge
Menu	One column of dishes	Two columns (from 768 pixels wide)
Section bar	Swipe sideways	Swipe or scroll sideways, or use the arrows at its edges
Cart	A bar along the bottom of the menu opens it. The panel fills the screen.	The Cart button opens it. The panel is a strip down the right side.

Part of the app	On a phone	On a wider screen
Checkout	The form, then the order summary	Form and summary side by side. The summary stays in view as you scroll.
Tracking	The status, then the order summary	Status and summary side by side
Kitchen board	Each order stacked: who, what, buttons	Each order on one row of three columns

1.13 Accessibility

- **Keyboard.** Everything can be reached and used with the keyboard alone. The control that has focus always shows the same green outline. The first press of Tab on any page offers "Skip to content", which jumps past the top bar.
- **Screen readers.** Every control has a name. Buttons that show only an icon on a phone still announce "Orders", "Sign out" and so on. The quantity buttons name their dish, for example "Add one more Masala Fries". The cart button announces how many items are inside.
- **Form errors** are announced as they appear and are tied to their field, so a screen reader reads the problem together with the field's label.
- **Nothing relies on colour alone.** Order statuses are always written out. The timeline also says "Completed", "In progress" or "Not started" for each stage. Vegetarian and non-vegetarian marks differ in shape as well as colour.
- **The cart panel** keeps focus inside it while it is open, closes with Escape, and returns focus to the button that opened it.
- **Motion.** The pulse on the current stage, the slide of the cart panel and smooth scrolling are all switched off for people who have asked their device to reduce motion.
- **Text contrast.** Body text is near-black on white. Text placed on gold is always dark, never white.

1.14 The full menu

43 dishes: 30 vegetarian, 13 non-vegetarian. Prices run from ₹59 to ₹389.

Pizza (4)

Dish	Type	Price	What it is
Margherita Pizza	Veg	₹249	Slow-fermented dough, San Marzano tomato, fresh mozzarella and torn basil.
Chicken Tikka Pizza	Non-veg	₹349	Tandoor-charred chicken tikka, red onion and coriander on a makhani base.
Pepperoni Pizza	Non-veg	₹379	Double pepperoni that crisps at the edges, over tomato and mozzarella.
Four Cheese Pizza	Veg	₹329	Mozzarella, cheddar, parmesan and a little blue cheese on a garlic base.

Burgers (4)

Dish	Type	Price	What it is
Double Smash Burger	Non-veg	₹289	Two smashed patties, melted cheddar, pickles and house sauce in a butter-toasted bun.
Crispy Chicken Burger	Non-veg	₹259	Buttermilk-fried chicken thigh, crunchy slaw and chilli mayo.
Classic Cheeseburger	Non-veg	₹229	A single patty with cheese, lettuce, tomato and onion on a sesame bun.
Garden Veggie Burger	Veg	₹219	A crisp vegetable and quinoa patty with avocado, tomato and rocket.

Indian Mains (5)

Dish	Type	Price	What it is
Hyderabadi Chicken Biryani	Non-veg	₹319	Long-grain basmati and chicken cooked dum-style with saffron, mint and fried onion.
Butter Chicken with Naan	Non-veg	₹349	Chicken in a smooth tomato and butter gravy, with two garlic naan.
Paneer Butter Masala	Veg	₹289	Soft paneer in a mildly spiced tomato and cashew gravy, served with jeera rice.
Tandoori Chicken	Non-veg	₹339	Half a chicken marinated overnight in yoghurt and spices, roasted until charred.
North Indian Veg Thali	Veg	₹229	Two rotis with mixed vegetable curry, dal tadka and pickle, served on a steel thali.

South Indian (2)

Dish	Type	Price	What it is
Masala Dosa	Veg	₹149	A crisp rice crepe filled with spiced potato, served with sambar and chutneys.
Idli Vada Sambar	Veg	₹129	Steamed idlis and a crisp medu vada with sambar and coconut chutney.

Chinese (3)

Dish	Type	Price	What it is
Veg Hakka Noodles	Veg	₹189	Wok-tossed noodles with cabbage, carrot and spring onion in soy and chilli.
Veg Fried Rice	Veg	₹179	Rice stir-fried over high heat with beans, carrot and spring onion.
Steamed Veg Momos	Veg	₹139	Eight dumplings filled with cabbage and carrot, with a fiery tomato dip.

Pasta (2)

Dish	Type	Price	What it is
Penne Arrabbiata	Veg	₹249	Penne in a tomato sauce with garlic, chilli and basil.
Prawn Spaghetti Aglio e Olio	Non-veg	₹389	Spaghetti tossed with prawns, garlic, chilli and cherry tomatoes in olive oil.

Wraps and Sandwiches (3)

Dish	Type	Price	What it is
Grilled Chicken Wrap	Non-veg	₹219	Spiced grilled chicken, crunchy slaw and garlic sauce rolled in a soft tortilla.
Chickpea Tacos	Veg	₹199	Three soft tacos with spiced chickpeas, avocado, tomato and a squeeze of lime.
Grilled Cheese Sandwich	Veg	₹149	Three cheeses melted between thick slices of bread, grilled in butter.

Salads and Bowls (2)

Dish	Type	Price	What it is
Chickpea and Avocado Salad	Veg	₹239	Chickpeas, avocado, cherry tomatoes, roasted sweet potato and greens with a tahini dressing.
Grilled Chicken Protein Bowl	Non-veg	₹279	Grilled chicken, boiled egg, sweetcorn, edamame and crunchy vegetables over greens.

Sides (5)

Dish	Type	Price	What it is
Masala Fries	Veg	₹99	Skin-on fries tossed in chaat masala, with garlic dip.
Hot Chicken Wings	Non-veg	₹229	Six wings glazed in a sticky chilli sauce, with a cooling ranch dip.
Punjabi Samosa	Veg	₹59	Two flaky samosas filled with spiced potato and peas, with mint chutney.
Paneer Tikka	Veg	₹249	Paneer, peppers and onion marinated in spiced yoghurt and grilled on skewers.
Loaded Chicken Nachos	Non-veg	₹249	Tortilla chips under melted cheese, spiced chicken, salsa, jalapeños and sour cream.

Desserts (6)

Dish	Type	Price	What it is
Chocolate Brownie	Veg	₹149	Dense and fudgy, baked with dark chocolate and walnuts.
Glazed Doughnuts	Veg	₹129	Three fresh doughnuts with chocolate glaze and sprinkles.
Blueberry Cheesecake	Veg	₹189	Baked cheesecake on a biscuit base, topped with blueberry compote.
Belgian Waffles	Veg	₹179	Two crisp waffles with blueberries and maple syrup.
Banana Pancakes	Veg	₹169	A tall stack of pancakes with sliced banana and warm maple syrup.
Strawberry Ice Cream Cone	Veg	₹89	Two scoops of strawberry ice cream in a crisp waffle cone.

Drinks (7)

Dish	Type	Price	What it is
Iced Coffee	Veg	₹139	Cold-brewed coffee over ice with a splash of milk.
Fresh Lime Soda	Veg	₹79	Squeezed lime, soda and mint. Sweet, salted or both.
Oreo Milkshake	Veg	₹169	Thick vanilla shake blended with Oreo and chocolate sauce.
Fresh Orange Juice	Veg	₹99	Oranges squeezed to order, with nothing added.
Strawberry Milkshake	Veg	₹159	Fresh strawberries blended with milk and vanilla ice cream.
Cappuccino	Veg	₹129	A double espresso under steamed milk and a thick layer of foam.
Mixed Berry Smoothie	Veg	₹179	Strawberries, blueberries and banana blended with yoghurt.

1.15 What the demo does not do

So that nothing is a surprise:

- **No payment.** No card details are asked for and nothing is charged.
- **No real delivery.** The kitchen and the rider are simulated by a clock.
- **No editing the menu from the site.** The menu is maintained by the developers and loaded into the database (see 2.13).
- **One kitchen.** There are no restaurants to choose between.
- **No notifications.** Updates appear on the tracking page. No email, SMS or push message is sent.
- **No saved addresses.** The address and phone number are typed in for each order.
- **No changing an order once placed.** It can be cancelled before the kitchen starts, and placed again.
- **No password reset and no changing your account details.**

Part 2: Technical documentation

2.1 What was asked and what was built

The assessment brief asks for an order management feature for a food delivery app: a menu, a cart with quantities, a checkout that collects delivery details, and an order status that updates in real time, with the updates simulated in the back end. It asks for a REST API, storage in memory or a simple database, a React front end built with Vite or Next.js, and tests written test-first for the API endpoints and the key UI components. It is judged on problem-solving, code quality, UI and UX, back-end security and validation, and the use of AI.

The brief asks for	What was built	Where
A menu showing name, description, price and image	A menu page with 43 dishes in 11 filterable sections, and `GET /api/v1/menu`	`src/app/page.tsx`, `src/features/menu/`, `src/server/modules/menu/`
Adding items to a cart and choosing quantities	A cart with add, increase, decrease and remove, capped at 20 of a dish, remembered across reloads	`src/features/cart/`
A checkout with name, address and phone number	A form validated in the browser and again by the API, with one shared set of rules	`src/features/checkout/`, `src/shared/schemas.ts`
Order status: Order Received, Preparing, Out for Delivery	Five statuses (those three, plus Delivered and Cancelled) and one rule set for the legal moves between them	`src/shared/order-status.ts`
Real-time updates, simulated in the back end	A simulated kitchen driven by timestamps, pushed to the browser over Server-Sent Events	`src/server/modules/orders/`
A REST API to manage orders and the menu	12 endpoints under `/api/v1`	`src/app/api/v1/`
Storage in memory or a simple database	MongoDB, the only external service	`src/server/models/`
TDD tests for the API and key UI components	339 tests. The git history shows failing-test commits ahead of the code that passes them.	`tests/`, and `*.test.ts` files beside the code
React with Vite or Next.js	Next.js 16 (App Router) with React 19	
A hosted app	Deployed on Vercel	`vercel.json`

Built beyond the brief, because an order system is not credible without them: accounts with JWT access and refresh tokens, a staff role with a kitchen board, cancellation, rate limiting, and a CI pipeline.

Two constraints shaped the design more than anything else:

1. **The app must run on Vercel.** Vercel runs each request in a short-lived function. No process stays up to run timers or hold connections, and function instances share no memory.
2. **MongoDB is the only external service.** No Redis, no message broker, no hosted realtime service.

Sections 2.9 and 2.10 explain how real-time updates work within both.

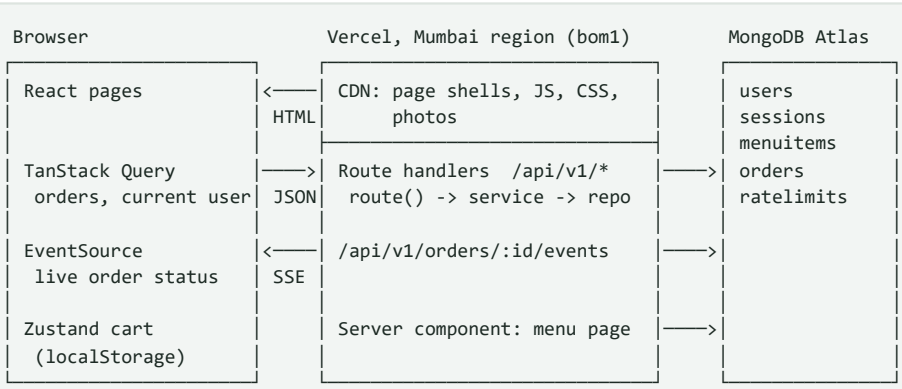
2.2 Technology stack

Area	Choice	What it does here
Framework	Next.js 16.4 (App Router), React 19.3	One codebase serves the pages and the REST API, and deploys to Vercel as a unit.

Area	Choice	What it does here
Language	TypeScript 5.9, strict mode	Types are shared by the browser and the server, so the two cannot drift apart unnoticed.
Styling	Tailwind CSS 4.3	Design tokens are declared once in CSS and used as utility classes.
Database	MongoDB Atlas, through Mongoose 9.11	Orders must survive between serverless invocations, which rules out process memory.
Validation	Zod 4.6	The same schemas validate the forms in the browser and the request bodies in the API.
Authentication	jose 6.2 (JWT, HS256), bcryptjs 3.0	Signed access and refresh tokens, and password hashing. Neither needs a native build step.
Server data in the browser	TanStack Query 5	Caches API data, refetches it, and lets pushed order updates replace cached ones.
Client state	Zustand 5	The cart: a small store saved to `localStorage`.
Forms	react-hook-form 7.90	Form state and focus management. Validation is handed to Zod through a small in-house resolver.
Accessible primitives	Radix Dialog 1.3	The cart panel's focus trap, Escape handling and focus return.
Icons and class utilities	lucide-react, class-variance-authority, clsx, tailwind-merge	Icons, typed component variants, and safe merging of class names.
Tests	Vitest 5, React Testing Library 16, jsdom 29, mongodb-memory-server 11	Unit, API and UI tests. The API tests run against a real MongoDB held in memory.
Tooling	ESLint 9, Prettier 3, tsx 4	Lint, formatting, and running the seed script.
Runtime	Node.js 22.12 or newer	Required by the test toolchain. Declared in `package.json` under `engines`.
Hosting and CI	Vercel, GitHub Actions	Every push to `main` is checked by CI and deployed to production.

2.3 Architecture

One Next.js application serves both the pages and the API. MongoDB is the only other moving part.



Three kinds of traffic leave the browser:

- **Pages.** The static parts come from Vercel's CDN. The menu is the one piece of page content rendered on the server for each request, and it is read straight from the database by a server component.

- **API calls.** Everything else the browser needs (who is signed in, orders, placing and cancelling) goes through the REST API as JSON.
- **The live stream.** The tracking page holds one Server-Sent Events connection open and receives the order whenever it changes.

Layers

A request to the API passes through three layers, each with one job.

1. **Route handler** (`src/app/api/v1/**/route.ts`). A few lines that declare who may call the endpoint and what the body must look like, call a service, and return the result. Every handler is built with one wrapper, `route()`, described in 2.4.
2. **Service** (`src/server/modules/**`). The business rules: who may see an order, which status changes are legal, what an order costs.
3. **Repository** (`src/server/modules/orders/order.repository.ts`). The only code that builds database queries for orders. Everything above it works with plain objects.

Beside those sits `src/shared/`, which both the browser and the server import. It is what keeps the two sides honest with each other:

- the checkout form and the API validate with the **same** Zod schema;
- the cart preview and the server price an order with the **same** function;
- the status timeline, the kitchen board's buttons and the API decide what is legal with the **same** transition rules.

What may import what

Folder	May import	Must not import
`src/shared/`	Nothing from the app	`server`, `features`, `lib`
`src/server/`	`shared`	`features`, `components`, `lib`
`src/features/`	`shared`, `lib`, `components`, other features	`server`
`src/components/`	`shared`, `features` (the header only)	`server`
`src/app/`	Anything. Pages import features; route handlers import `server`.	

The modules that hold the environment configuration and the database connection start with `import "server-only"`. If browser code ever imported them, directly or through something that uses them, the build would fail rather than ship server code to the browser.

Folder structure

```
src/
  app/                          Routes: pages and API handlers, kept thin
    page.tsx                    Menu page (static banner, streamed menu)
    checkout/  login/  register/
    orders/  orders/[id]/  admin/orders/
    api/v1/                     REST endpoints
      menu/route.ts                   GET
      orders/route.ts                 POST, GET
      orders/[id]/route.ts            GET, DELETE
      orders/[id]/status/route.ts     PATCH
      orders/[id]/events/route.ts     GET (Server-Sent Events)
      auth/register|login|refresh|logout|me/route.ts
    layout.tsx  globals.css     Page frame, fonts, design tokens
    error.tsx  not-found.tsx    Failure and 404 screens
    icon.png  apple-icon.png  opengraph-image.png
  features/                     Browser-side features, one folder each
```

```
    menu/  cart/  checkout/  orders/  auth/  admin/
  components/
    ui/                          Reusable primitives: Button, TextField, Sheet...
    site-header.tsx  page-shell.tsx  providers.tsx
  lib/                           Browser helpers: API client, formatting, form resolver
  shared/                        Used by BOTH browser and server
    order-status.ts              Statuses and the rules for moving between them
    pricing.ts  money.ts         What an order costs, and how money is shown
    schemas.ts                   Zod validation for every request body
    types.ts                     The shapes that cross the network
    demo-accounts.ts
  server/                        Server-only code
    http/                        route() wrapper, errors, responses, cookies, rate limit
    modules/
      auth/                      Tokens, password hashing, sessions
      menu/                      Catalogue, seeding, listing
      orders/                    Service, repository, simulation, event stream
    models/                      Mongoose schemas for the five collections
    db/connect.ts                Cached MongoDB connection
    config/env.ts                Validated environment variables
tests/
  api/                           Endpoint tests against an in-memory MongoDB
  helpers/                       Test client, factories, fixtures, fake EventSource
  setup/                         Setup files for each test project
scripts/seed.ts                  Loads the menu and the demo accounts
public/
  menu/                          43 dish photos
  brand/                         Logo files
```

2.4 How a request is handled

Every endpoint is built with one function, `route()`, in `src/server/http/route.ts`. A route file states what the endpoint needs and supplies only its own logic. This is the whole of the endpoint that places an order:

```
// src/app/api/v1/orders/route.ts
export const POST = route(
  { auth: "user", body: createOrderSchema, rateLimit: ORDER_RATE_LIMIT },
  async ({ user, body }) => {
    return created({ order: await placeOrder(user, body) });
  },
);
```

The options:

Option	Value	Meaning
`auth`	left out	Public. Nobody is identified.
`auth`	`"user"`	The caller must be signed in.
`auth`	`"admin"`	The caller must be signed in as kitchen staff.
`auth`	`"optional"`	The caller is identified if signed in. A visitor passes through as `null`.
`body`	a Zod schema	The JSON body is parsed and validated before the handler runs. The handler gets typed, clean data.
`rateLimit`	a rule	An allowance of requests per network address.

TypeScript overloads make `user` non-null inside the handler exactly when `auth` is `"user"` or `"admin"`. A handler cannot forget to check whether someone is signed in, because the type will not let it read `user.userId` otherwise.

The wrapper runs these steps in order. A failure at any step stops the request there.

Step	What happens	If it fails
1	**Origin check.** A POST, PATCH or DELETE that carries an `Origin` header naming another host is refused.	`403 FORBIDDEN`
2	**Database connection.** Opened on the first request to a function instance and reused after that.	`500 INTERNAL_ERROR`
3	**Rate limit,** if the endpoint declares one.	`429 RATE_LIMITED` with a `Retry-After` header
4	**Identify the caller,** if `auth` is set: read the `fo_access` cookie and verify the token.	`401 UNAUTHENTICATED`, or `403 FORBIDDEN` for a wrong role
5	**Parse and validate the body,** if a schema is given.	`400 BAD_REQUEST` or `400 VALIDATION_ERROR`
6	**Read the route parameters,** such as the order id.	
7	**Run the handler.**	Whatever the service throws

Three things about that order are deliberate:

- The rate limit runs before authentication and before password hashing, so a flood of requests is turned away before it costs anything. The side effect is that the allowance is counted per network address, whoever is or is not signed in.
- Identifying the caller needs no database query. The access token is verified by its signature alone.
- A customer who calls a staff endpoint is refused at step 4, before the body is even read.

Every failure is a thrown error, and one `catch` deals with all of them:

- Errors that Next.js throws for its own control flow are passed straight back to the framework (`unstable_rethrow`). Without this, a production build logged a false 500 for a handler it had deliberately stopped prerendering.
- An `AppError` (`src/server/http/errors.ts`) is one that is safe to show. It carries a status, a code, a message, and optionally per-field details and response headers, and it becomes the JSON error response.
- Anything else is a bug. The real cause is written to the server log, and the client receives only `500 INTERNAL_ERROR` with "Something went wrong on our side".

2.5 Data model

Five MongoDB collections, each defined by a Mongoose schema in `src/server/models/`.

`users`

Field	Type	Notes
`name`	string	Trimmed
`email`	string	Lower-cased and trimmed. **Unique index.**
`passwordHash`	string	bcrypt hash. The password itself is never stored.
`role`	string	`customer` or `admin`. Defaults to `customer`.
`createdAt`, `updatedAt`	date	Maintained by Mongoose

The unique index is what prevents duplicate accounts. A "check, then insert" in application code would let two simultaneous sign-ups with one email both pass the check. Instead the insert is attempted, and a duplicate-key error from MongoDB becomes `409 EMAIL_TAKEN`.

`sessions`

One document for each sign-in. It records which refresh token is the current one, which is what makes sign-out and token rotation real.

Field	Type	Notes
`userId`	ObjectId	Indexed
`tokenHash`	string	SHA-256 of the current refresh token's id. The token itself is never stored.
`expiresAt`	date	**TTL index.** MongoDB deletes the session by itself once this time has passed.
`createdAt`, `updatedAt`	date	

`menuitems`

Field	Type	Notes
`name`	string	
`slug`	string	A stable identifier such as `margherita-pizza`. **Unique index.**
`description`	string	
`price`	number	An integer number of paise
`image`	string	A path under `/menu/`, served from `public/menu/`
`category`	string	One of the 11 sections in `MENU_CATEGORIES`
`isVeg`	boolean	
`isAvailable`	boolean	A dish taken off the menu is switched off here, never deleted, because orders refer to it
`sortOrder`	number	Position within its section

`orders`

Field	Type	Notes
`userId`	ObjectId	The customer who placed it
`items[]`	array	`menuItemId`, `name`, `unitPrice`, `quantity`
`customer`	object	`name`, `address`, `phone`, as given at checkout
`subtotal`, `deliveryFee`, `tax`, `total`	number	Integers in paise, computed by the server
`status`	string	`received`, `preparing`, `out_for_delivery`, `delivered` or `cancelled`
`statusHistory[]`	array	`status` and `at` for every status the order has been in
`nextTransitionAt`	date or null	When the simulated kitchen is due to move the order on. Null once it is delivered or cancelled.
`createdAt`, `updatedAt`	date	Set explicitly by the code, not by Mongoose

Three decisions are visible in that shape:

- **Each item is a snapshot.** The dish's name and price are copied into the order when it is placed. Editing or re-pricing the menu later never rewrites what a past order says or cost.

- **The history is stored, not derived.** Each status change appends to `statusHistory`, which is what the timeline on the tracking page draws.
- **Timestamps are set by hand** so that a status change, its history entry and `updatedAt` always carry exactly the same instant.

Indexes:

Index	Serves
`{ userId: 1, createdAt: -1 }`	"My orders, newest first"
`{ status: 1, createdAt: -1 }`	The kitchen's newest-first list, filtered by status
`{ status: 1, nextTransitionAt: 1 }`	Finding the orders whose next simulated step is due

An order document looks like this (the ids are illustrative):

```
{
  "_id": "6acf2b1e9d3a4c7f8e1b2a30",
  "userId": "6ace6a7e2645ab7a662f6daa",
  "items": [
    {
      "menuItemId": "6ace6a7d2645ab7a662f6d81",
      "name": "Margherita Pizza",
      "unitPrice": 24900,
      "quantity": 1
    },
    {
      "menuItemId": "6ace6a7d2645ab7a662f6d99",
      "name": "Masala Fries",
      "unitPrice": 9900,
      "quantity": 2
    }
  ],
  "customer": {
    "name": "Asha Rao",
    "address": "12 MG Road, Indiranagar, Bengaluru 560038",
    "phone": "+919876543210"
  },
  "subtotal": 44700,
  "deliveryFee": 4000,
  "tax": 2235,
  "total": 50935,
  "status": "preparing",
  "statusHistory": [
    { "status": "received", "at": "2026-10-14T13:30:00.000Z" },
    { "status": "preparing", "at": "2026-10-14T13:30:20.000Z" }
  ],
  "nextTransitionAt": "2026-10-14T13:30:40.000Z",
  "createdAt": "2026-10-14T13:30:00.000Z",
  "updatedAt": "2026-10-14T13:30:20.000Z"
}
```

`ratelimits`

A request counter for one client, one rule and one time window.

Field	Type	Notes
`key`	string	`<rule>:<address>:<window start>`, for example `auth:203.0.113.7:1791918420000`. **Unique index.**
`count`	number	Requests seen in this window
`expiresAt`	date	**TTL index.** The counter deletes itself when its window is over.

2.6 API reference

Conventions

- **Base path:** `/api/v1`, on the same origin as the site. On the live site that is `https://atithya-kitchen.vercel.app/api/v1`.
- **Format:** JSON in both directions. A request with a body sends `Content-Type: application/json`.
- **Success:** `{ "data": ... }`.
- **Failure:** `{ "error": { "code": "...", "message": "...", "details": { ... } } }`. `details` is present only for validation errors.
- **Authentication:** two httpOnly cookies set by the sign-in and sign-up endpoints. There is no `Authorization` header. A browser sends the cookies by itself; a command-line client needs a cookie jar.
- **Money:** integers in paise. ₹249 is `24900`.
- **Dates:** ISO 8601 strings in UTC.
- **Ids:** 24-character hexadecimal strings (MongoDB ObjectIds).
- **Unknown fields** in a request body are dropped, not rejected.

Endpoints at a glance

Method	Path	Who	Purpose
`GET`	`/menu`	Anyone	The dishes on the menu, in menu order
`POST`	`/orders`	Signed in	Place an order
`GET`	`/orders`	Signed in	Your own orders. Staff get everyone's.
`GET`	`/orders/:id`	The order's customer, or staff	One order
`DELETE`	`/orders/:id`	The order's customer, or staff	Cancel the order
`PATCH`	`/orders/:id/status`	Staff	Move the order to its next status, or cancel it
`GET`	`/orders/:id/events`	The order's customer, or staff	A live stream of the order
`POST`	`/auth/register`	Anyone	Create an account and sign in
`POST`	`/auth/login`	Anyone	Sign in
`POST`	`/auth/refresh`	Whoever holds a refresh token	Renew the session
`POST`	`/auth/logout`	Anyone	End the session
`GET`	`/auth/me`	Anyone	The signed-in user, or `null`

Error codes

HTTP	code	When
400	`BAD_REQUEST`	The body is not valid JSON
400	`VALIDATION_ERROR`	A field is missing or wrong. `details` maps each field's path to its messages.
401	`UNAUTHENTICATED`	No valid access token, or a refresh token that no longer works
401	`INVALID_CREDENTIALS`	Wrong email or password at sign-in
403	`FORBIDDEN`	Signed in but not allowed (a customer on a staff endpoint), or a request from another website
404	`NOT_FOUND`	The order does not exist, its id is malformed, or it belongs to someone else

HTTP	code	When
409	`EMAIL_TAKEN`	Signing up with an email that already has an account
409	`INVALID_STATUS_TRANSITION`	The order's current status does not allow the requested move, or someone else changed the order first
409	`ORDER_NOT_CANCELLABLE`	Cancelling an order that is past the point where the caller may cancel it
429	`RATE_LIMITED`	Too many requests from one address. The `Retry-After` header gives the seconds to wait.
500	`INTERNAL_ERROR`	An unexpected failure. The cause is logged on the server and never sent to the client.

A validation error names every field that is wrong, all at once:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some fields are invalid",
    "details": {
      "customer.phone": ["Enter a valid phone number (10 to 15 digits)"],
      "items.0.quantity": ["Quantity must be at least 1"]
    }
  }
}
```

Shapes

These are the types in `src/shared/types.ts`. The API returns them and the browser consumes them, so both sides are checked against one definition.

```
type UserDTO = {
  id: string;
  name: string;
  email: string;
  role: "customer" | "admin";
};

type MenuItemDTO = {
  id: string;
  name: string;
  description: string;
  price: number; // paise
  image: string;
  category: MenuCategory; // "pizza" | "burgers" | "indian" | ...
  isVeg: boolean;
};

type OrderDTO = {
  id: string;
  userId: string;
  items: {
    menuItemId: string;
    name: string;
    unitPrice: number;
    quantity: number;
    lineTotal: number;
  }[];
  customer: { name: string; address: string; phone: string };
  subtotal: number;
  deliveryFee: number;
  tax: number;
  total: number;
  status: OrderStatus; // "received" | "preparing" | "out_for_delivery" | "delivered" | "cancelled"
  statusHistory: { status: OrderStatus; at: string }[];
  estimatedDeliveryAt: string | null; // null once delivered or cancelled
  createdAt: string;
```

```
  updatedAt: string;
};
```

`lineTotal` and `estimatedDeliveryAt` are not stored. They are worked out each time an order is read.

Menu

`GET /menu` is public.

Returns `200` with `{ "data": { "items": MenuItemDTO[] } }`. Only dishes that are available are included, sorted by section, then by position within the section, then by name.

The site's own menu page does not call this endpoint. It reads the same data through the same function, `listMenu()`, from a server component. The endpoint exists for other clients of the API.

Orders

`POST /orders` needs a signed-in caller. It is rate limited to 10 requests a minute per address.

Request body:

```
{
  "items": [
    { "menuItemId": "6ace6a7d2645ab7a662f6d81", "quantity": 1 },
    { "menuItemId": "6ace6a7d2645ab7a662f6d99", "quantity": 2 }
  ],
  "customer": {
    "name": "Asha Rao",
    "address": "12 MG Road, Indiranagar, Bengaluru 560038",
    "phone": "+91 98765 43210"
  }
}
```

The body carries ids and quantities only. A price, a total or a status sent by the client is dropped before the handler sees it.

Returns `201` with `{ "data": { "order": OrderDTO } }`. The order starts as `received`.

Failure	Response
Not signed in	`401 UNAUTHENTICATED`
Empty cart, bad quantity, incomplete details	`400 VALIDATION_ERROR`, with keys such as `items`, `items.0.quantity`, `customer.address`
The same dish listed twice	`400 VALIDATION_ERROR` on `items.<n>.menuItemId`: "This item is listed more than once"
A dish that is not on the menu, or is unavailable	`400 VALIDATION_ERROR` on `items.<n>.menuItemId`: "This item is no longer available"
More than 10 orders a minute from one address	`429 RATE_LIMITED`

`GET /orders` needs a signed-in caller.

Returns `200` with `{ "data": { "orders": OrderDTO[] } }`, newest first, at most 50. A customer gets their own orders. Staff get every customer's.

An optional `?status=` filter takes one of the five statuses, for example `/orders?status=preparing`. Any other value is `400 VALIDATION_ERROR` on `status`.

`GET /orders/:id` needs a signed-in caller who is the order's customer, or staff.

Returns `200` with `{ "data": { "order": OrderDTO } }`.

Returns `404 NOT_FOUND` in three cases that are deliberately indistinguishable: no order has that id, the id is not a valid id, or the order belongs to another customer. Answering "forbidden" for the last case would confirm that the id exists.

`DELETE /orders/:id` cancels the order. It needs a signed-in caller who is the order's customer, or staff.

Orders are never erased, so that customers and staff keep a full history. `DELETE` is the only way an order can be withdrawn, and what it does is set the status to `cancelled`.

Returns `200` with the cancelled order.

Failure	Response
A customer cancelling once the order is `preparing` or later	`409 ORDER_NOT_CANCELLABLE`
Staff cancelling once the order is `out_for_delivery` or later	`409 ORDER_NOT_CANCELLABLE`
Cancelling an order that is already cancelled	`409 ORDER_NOT_CANCELLABLE`
Another customer's order, or an unknown id	`404 NOT_FOUND`

`PATCH /orders/:id/status` is for staff.

Request body: `{ "status": "preparing" }`. The status must be the next one in sequence, or `cancelled` while the order is `received` or `preparing`.

Returns `200` with the updated order.

Failure	Response
Called by a customer, even for their own order	`403 FORBIDDEN`
Skipping a step, moving backwards, or any move out of a finished order	`409 INVALID_STATUS_TRANSITION`, for example `An order that is "Order Received" cannot be moved to "Delivered"`
The order changed between being read and being written	`409 INVALID_STATUS_TRANSITION`: "This order was just updated. Refresh and try again."
A status that does not exist	`400 VALIDATION_ERROR` on `status`
An unknown id	`404 NOT_FOUND`

`GET /orders/:id/events` is a Server-Sent Events stream. It needs a signed-in caller who is the order's customer, or staff.

Access is checked before the stream starts, so a missing or forbidden order is an ordinary JSON `404`, and a signed-out caller an ordinary `401`, rather than a stream that fails silently.

The response has `Content-Type: text/event-stream`, `Cache-Control: no-cache, no-transform` and `X-Accel-Buffering: no`. The stream sends the order at once, sends it again each time it changes, and ends after sending a delivered or cancelled order:

```
retry: 2000

event: order
data: {"id":"6acf2b1e9d3a4c7f8e1b2a30","status":"received", ...}

: keep-alive
```

```
event: order
data: {"id":"6acf2b1e9d3a4c7f8e1b2a30","status":"preparing", ...}
```

Each `data:` line is a complete `OrderDTO`. Section 2.10 describes the stream in full.

Authentication

`POST /auth/register` is public and rate limited (10 a minute per address, shared with sign-in).

Request body: `{ "name": "Asha Rao", "email": "asha@example.com", "password": "at least 8 characters" }`.

Returns `201` with `{ "data": { "user": UserDTO } }` and sets both session cookies, so the new account is signed in. The response never contains the password or its hash. A `role` sent in the body is ignored: every account created here is a customer.

Fails with `400 VALIDATION_ERROR`, `409 EMAIL_TAKEN` or `429 RATE_LIMITED`.

`POST /auth/login` is public and rate limited.

Request body: `{ "email": "...", "password": "..." }`.

Returns `200` with `{ "data": { "user": UserDTO } }` and sets both session cookies.

Fails with `401 INVALID_CREDENTIALS` for a wrong password and for an unknown email alike, with the same message and after the same amount of work.

`POST /auth/refresh` takes no body. It reads the `fo_refresh` cookie.

Returns `200` with `{ "data": { "user": UserDTO } }` and replaces both cookies. The refresh token it was given stops working.

Fails with `401 UNAUTHENTICATED` and clears both cookies if the token is missing, expired, already used, or belongs to a session that has ended.

`POST /auth/logout` takes no body.

Deletes the session and clears both cookies. Returns `200` with `{ "data": { "signedOut": true } }`, including when nobody was signed in.

`GET /auth/me` answers "who is signed in?" and has three outcomes:

Situation	Response	What the browser does
A valid access token	`200` `{ "data": { "user": UserDTO } }`	Shows the signed-in interface
No session at all	`200` `{ "data": { "user": null } }`	Shows the visitor's interface
No usable access token, but a refresh cookie is present	`401 UNAUTHENTICATED`	Refreshes the session and asks again

Being signed out is a normal state, not an error. Answering a visitor with `200` and `null` keeps a failed request out of the browser's console on every page load.

Trying it from a terminal

The demo customer's credentials are public by design (see 2.13). In a POSIX shell such as Git Bash, macOS or Linux:

```
BASE=https://atithya-kitchen.vercel.app/api/v1

# Sign in. -c saves the session cookies to a file.
curl -c cookies.txt -H "content-type: application/json" \
```

```
  -d '{"email":"asha@atithya.example","password":"demo-customer-1"}' \
  $BASE/auth/login

# The menu is public. Copy an "id" from the reply.
curl $BASE/menu

# Place an order. -b sends the saved cookies.
curl -b cookies.txt -H "content-type: application/json" \
  -d '{"items":[{"menuItemId":"PASTE_A_MENU_ID","quantity":2}],"customer":{"name":"Asha Rao","address":"12 MG Road,
Indiranagar, Bengaluru 560038","phone":"+91 98765 43210"}}' \
  $BASE/orders

# Watch it move. -N prints each event as it arrives.
curl -N -b cookies.txt $BASE/orders/PASTE_THE_ORDER_ID/events
```

2.7 Authentication and sessions

The two tokens

Signing in issues two JSON Web Tokens, signed with HS256 using the `jose` library.

	Access token	Refresh token
Purpose	Proves who is calling, on every API request	Obtains a new access token when the old one expires
Lifetime	15 minutes	7 days
Signed with	`JWT_ACCESS_SECRET`	`JWT_REFRESH_SECRET`
Audience claim	`access`	`refresh`
Carries	The user's id (`sub`) and `role`	The user's id (`sub`), session id (`sid`), token id (`jti`)
Cookie	`fo_access`	`fo_refresh`
Cookie path	`/`	`/api/v1/auth`

Each token type has its own secret and its own audience, so one can never be accepted in place of the other. The environment check refuses to start with two identical secrets. There are tests for both directions: a refresh token cannot be used as an access token, and an access token cannot be used to refresh.

A short access token limits the damage if one leaks. The long refresh token is what lets people stay signed in for a week without that risk, because it is used rarely, sent only to the auth endpoints, and can be revoked.

The cookies

Both cookies are set with:

- `HttpOnly`, so no script running in the page can read them. A cross-site scripting bug could not steal the session.
- `SameSite=Lax`, so the browser does not attach them to requests made by other websites.
- `Secure` in production, so they are only ever sent over HTTPS.
- A `Max-Age` equal to the token's lifetime, so they outlive the browser being closed.

The refresh cookie's path is `/api/v1/auth`. The browser therefore sends it only to the five auth endpoints and never with ordinary API calls, which keeps the more valuable token off the wire nearly all the time.

Cookies were chosen over keeping tokens in `localStorage` and sending an `Authorization` header for two reasons. Script cannot read an httpOnly cookie. And the browser's `EventSource`, which carries the live stream, cannot set request headers but does send cookies.

Sessions in the database

A JWT cannot be taken back once issued. To make sign-out and theft detection real, every sign-in also creates a `sessions` document recording which refresh token is the session's current one. Only the SHA-256 hash of the token's id is stored, so a leaked database does not yield usable tokens.

Signing in

1. `POST /auth/login` arrives. The rate limit is checked, then the body is validated.
2. The user is looked up by email.
3. A bcrypt comparison always runs. If no user was found, the password is compared against a placeholder hash, so the response takes the same time whether or not the email has an account.
4. If either check fails, the answer is `401 INVALID_CREDENTIALS`, with nothing to tell the two cases apart.
5. A session document is created with a fresh random token id.
6. Both tokens are signed and set as cookies.

Signing up does the same from step 5, after inserting the user with a bcrypt hash (cost factor 10) and the role `customer`.

An ordinary request

`route()` reads `fo_access`, verifies its signature, issuer, audience and expiry, and takes the user's id and role from it. There is no database lookup. The services receive an `Actor`, `{ userId, role }`, that comes only from the verified token and never from the request body.

Refreshing, and catching a stolen token

When the access token expires, the next API call fails with `401 UNAUTHENTICATED`. The browser's API client then calls `POST /auth/refresh` and repeats the original request once.

The refresh token is **rotated**: each one works exactly once.

```
// src/server/modules/auth/auth.service.ts
const session = await SessionModel.findOneAndUpdate(
  {
    _id: claims.sessionId,
    userId: claims.userId,
    tokenHash: hashTokenId(claims.tokenId),
  },
  { tokenHash: hashTokenId(nextTokenId), expiresAt: sessionExpiry() },
);

if (!session) {
  await SessionModel.deleteOne({ _id: claims.sessionId });
  throw errors.unauthenticated();
}
```

The update matches only if the token presented is the session's current one, and in the same atomic step replaces it with the next. So:

- Of two simultaneous refreshes with the same token, only one can win.
- A token that has already been used matches nothing. A used token coming back means two parties hold it: the real user and whoever copied it. The server cannot tell which is which, so it deletes the whole session and both must sign in again. This is **reuse detection**.
- Every successful refresh pushes the session's expiry seven days ahead, which is why an active user stays signed in.

A failed refresh also clears both cookies, since a refresh token that no longer works is of no use to the browser.

Signing out

`POST /auth/logout` deletes the session document and clears both cookies. A refresh token copied before sign-out is useless afterwards, and there is a test for exactly that.

In the browser

`src/lib/api-client.ts` wraps `fetch`:

- When a request fails with `401 UNAUTHENTICATED`, it refreshes the session and repeats the request **once**.
- If several requests find the session expired at the same moment, they all wait on a single refresh. This matters because each refresh token works only once: two parallel refreshes would look like a replay and end the session.
- The sign-in, sign-up, refresh and sign-out endpoints are never retried this way. A `401` from sign-in means "wrong password", not "expired session".

Who may do what

Permissions are enforced in two places, both on the server:

- **At the route.** `auth: "admin"` refuses anyone whose token does not carry the `admin` role. The role lives inside the signed token and cannot be edited by its holder.
- **In the service.** Every read of an order goes through `loadVisibleOrder`, which returns the order only if the caller is staff or the order's own customer. Otherwise it reports "not found".

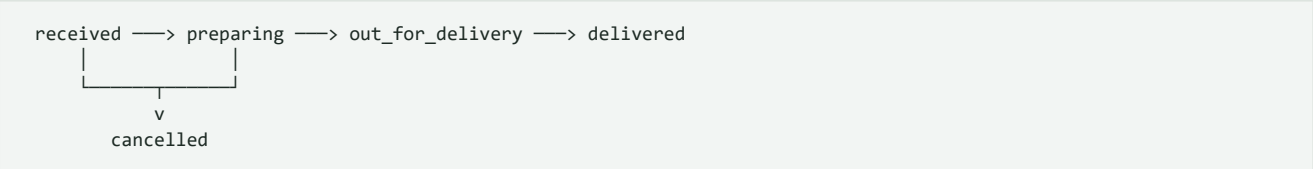
The pages add nothing to this. `/admin/orders` is a public page that shows "This page is for kitchen staff" to a customer. It protects no data by itself. The API does.

What follows from this design

- An access token cannot be revoked before it expires. After sign-out the browser no longer has it, but a copy made beforehand would work for the rest of its 15 minutes. The same window applies if an account's role is changed.
- If the JWT secrets are changed, every existing token stops verifying and everyone is signed out.

2.8 Order lifecycle

Statuses and legal moves



The rules are four small functions in `src/shared/order-status.ts`:

Function	Rule
`nextStatus(status)`	The next status in sequence, or `null` after `delivered` or `cancelled`
`canTransition(from, to)`	True for exactly one step forward, or for `cancelled` when `from` is `received` or `preparing`
`canCustomerCancel(status)`	True only for `received`
`isTerminal(status)`	True for `delivered` and `cancelled`

Put together, this is who can do what:

The order is	Its customer	Staff	The simulation
`received`	Cancel	Start preparing, or cancel	Moves it to `preparing`
`preparing`	Nothing	Send out for delivery, or cancel	Moves it to `out_for_delivery`
`out_for_delivery`	Nothing	Mark as delivered	Moves it to `delivered`
`delivered`	Nothing	Nothing	Nothing
`cancelled`	Nothing	Nothing	Nothing

Because these functions live in `shared`, the same code decides which buttons the kitchen board offers, whether the tracking page shows "Cancel order", and whether the API accepts the request. The interface cannot offer something the server would refuse for a reason the interface could have known.

Placing an order

`placeOrder` in `order.service.ts`:

1. Loads the menu items named in the request, in one query, keeping only those that are available.
2. If any requested item is missing from that result, refuses the whole order with a validation error that names each one by its position (`items.1.menuItemId`). The checkout page uses the position to tell the customer which dish it was.
3. Builds the order's lines from the **database's** name and price for each dish. Whatever the browser believed an item cost is never read.
4. Prices the order with `calculateTotals`, the same function the cart preview uses.
5. Inserts the order with status `received`, a history of one entry, and `nextTransitionAt` set one step ahead.

Changing a status safely

Two requests can try to change one order at the same moment: two members of staff, or staff and a customer's cancellation, or either and the simulation. The write is therefore conditional:

```
// src/server/modules/orders/order.repository.ts
return OrderModel.findOneAndUpdate(
  { _id: change.id, status: change.from },
  {
    $set: {
      status: change.to,
      updatedAt: change.at,
      nextTransitionAt: change.nextTransitionAt,
    },
    $push: { statusHistory: { status: change.to, at: change.at } },
  },
  { returnDocument: "after" },
).lean();
```

The filter includes the status the caller saw. If something else changed the order first, the filter matches nothing, nothing is written, and the service answers `409` with "This order was just updated. Refresh and try again." MongoDB applies an update to a single document atomically, so two concurrent changes can never both succeed.

A manual change also resets `nextTransitionAt` to one step from now, or to `null` if the order has finished. That is why moving an order by hand restarts the simulated countdown.

2.9 The simulated kitchen

The brief asks for status updates that are "simulated in the back end". The obvious way is a timer: when an order is placed, schedule three callbacks. On Vercel that cannot work. A function runs only while it is answering a request, so there is no process to hold a timer, and a scheduled job would be far too coarse for a 20-second step.

The solution is to **settle on read**. Nothing runs in the background. Instead:

- Each order stores `nextTransitionAt`, the moment its next status is due.
- Whenever an order is read, any steps that have fallen due are applied first.

The effect is the same as a timer would give, with nothing to keep running and nothing lost when an instance is recycled.

The length of a step comes from the `ORDER_STEP_SECONDS` environment variable, which defaults to 20.

Time since the order was placed	Status
0	`received`
1 step (20 s)	`preparing`
2 steps (40 s)	`out_for_delivery`
3 steps (60 s)	`delivered`

How overdue steps are applied

`settleDueOrders` runs one update for each of the three automatic steps, in sequence:

```
// src/server/modules/orders/order.repository.ts
for (const [from, to] of AUTOMATIC_STEPS) {
  const isLastStep = to === AUTOMATIC_STEPS.at(-1)![1];

  await OrderModel.updateMany(
    { ...toQuery(filter), status: from, nextTransitionAt: { $lte: now } },
    [
      {
        $set: {
          status: to,
          updatedAt: "$nextTransitionAt",
          statusHistory: {
            $concatArrays: [
              "$statusHistory",
              [{ status: to, at: "$nextTransitionAt" }],
            ],
          },
          nextTransitionAt: isLastStep
            ? null
            : { $add: ["$nextTransitionAt", stepMs] },
        },
      },
    ],
    { updatePipeline: true },
  );
}
```

Four properties come out of that one statement.

- **A step is applied exactly once.** The filter matches only orders still in the previous status. However many requests run this at the same moment, MongoDB lets one of them change a given order, and the rest find nothing left to match. A test fires many requests together and checks that the history has one entry for each step.
- **An order catches up in one call.** Because the three steps run in sequence, an order nobody looked at for ten minutes passes through `preparing`, `out_for_delivery` and `delivered` in a single read.

- **The timeline is truthful.** The update is a pipeline, so it can read the order's own `nextTransitionAt`. Each change is stamped with the time it was **due**, not the time someone happened to look. An order placed at 13:30:00 and first read at 13:35:00 shows 13:30:20, 13:30:40 and 13:31:00.
- **It is cheap.** Each step is one indexed update, served by the `{ status, nextTransitionAt }` index.

Where settling happens

Reading	What is settled
One order: fetching it, cancelling it, changing its status, or streaming it	That order, and only if it is actually due
A customer's list of orders	That customer's orders
The kitchen's list	Every order

Every rule check goes through `loadVisibleOrder`, which settles before it returns. A decision is therefore always made against the order's true current status. A customer cannot cancel an order whose 20 seconds have run out merely because nobody had looked at it since.

The arrival estimate

`estimateDelivery` takes `nextTransitionAt` and adds one step for each stage still to come after it. A new order is estimated three steps ahead. The estimate holds still while the order runs on schedule, moves earlier when staff move the order ahead, and is `null` once the order has finished.

2.10 Live updates

Server-Sent Events, not sockets

The app has no WebSocket server. Live updates use Server-Sent Events (SSE), a standard in which the browser opens one ordinary HTTP request and the server keeps the response open, writing an event to it whenever there is news.

SSE was chosen because it fits both the problem and the constraints:

- **Status tracking flows one way,** from server to browser. That is what SSE is for. Nothing the customer does needs to travel up the same connection.
- **It is plain HTTP.** A streamed response is something a Vercel function can produce. A socket server needs a process that stays up to hold connections, which Vercel's functions do not provide.
- **It needs no extra service.** The usual way to run sockets on a serverless host is a hosted realtime service, and this project's rule is that MongoDB is the only external service.
- **The browser does the hard parts.** `EventSource` reconnects by itself after a drop. It also sends cookies, so the httpOnly session works with no token handled in JavaScript.

The visible result is the same as with sockets: the status changes on screen without a reload.

The server side

`order-events.ts` builds the stream. Serverless instances share no memory, so there is no in-process event bus to subscribe to. The database is the one thing every instance can see. The stream therefore checks it on a short interval:

```
// src/server/modules/orders/order-events.ts (abridged)
send(`retry: ${RECONNECT_MS}\n\n`);

while (
  !signal.aborted &&
  !readerGone &&
  Date.now() - startedAt < maxDurationMs
```

```
) {
  const order = await loadOrder();
  const version = `${order.status}@${order.updatedAt}`;

  if (version !== lastSent) {
    send(`event: order\ndata: ${JSON.stringify(order)}\n\n`);
    lastSent = version;
    lastWriteAt = Date.now();
    if (isTerminal(order.status)) break;
  } else if (Date.now() - lastWriteAt >= KEEP_ALIVE_MS) {
    send(": keep-alive\n\n");
    lastWriteAt = Date.now();
  }

  await sleep(POLL_MS, signal);
}
```

Behaviour	Value	Why
How often it checks	Every 2 seconds	A change reaches the customer within about two seconds
What counts as a change	`status` plus `updatedAt`	The order is sent only when it differs from what was last sent
Keep-alive	A comment line after 15 quiet seconds	Proxies often close connections that look idle
Reconnect delay	`retry: 2000`	Tells the browser to wait two seconds before reconnecting
End of the order	The stream closes	Nothing more will happen to a delivered or cancelled order
Maximum length	240 seconds	The route allows itself 300 seconds (`maxDuration`). Closing sooner makes the end a clean one, and the browser reconnects.
The customer leaves	The loop stops at once	The request's abort signal ends the wait and the polling
The order cannot be loaded	The stream closes, without crashing	The browser reconnects, and its ordinary request for the order reports the cause

Two details tie this to the rest of the system:

- `loadOrder` is the same `getOrder` the ordinary endpoint uses. Reading the order applies any overdue simulation steps, which is exactly what makes the status move while a customer is watching.
- The same call re-checks permission on every poll, so a stream cannot outlive the caller's right to see the order.

The browser side

`use-order-stream.ts` opens an `EventSource` on the events endpoint while the order is in progress, and reports the connection's state:

State	When	The page says
`connecting`	The stream has been requested	"Connecting to live updates"
`live`	The connection is open	"Live updates on"
`reconnecting`	The connection dropped and the browser is retrying	"Reconnecting to live updates"
`paused`	The server refused the stream and the session could not be renewed	"Live updates are paused. Reload the page to reconnect."
`idle`	The order is delivered or cancelled, so nothing is listened for	Nothing

`EventSource` recovers from network drops by itself. What it cannot recover from is being refused outright, which is what happens when the access token has expired. In that case the hook renews the session and opens a new stream, at most twice in a row, before settling on `paused`.

Each pushed order is written into the TanStack Query cache under the same key the page's ordinary request uses. The page re-renders from the cache, so the headline, timeline, arrival time and cancel button all update from one assignment. The first paint never waits for the stream: it comes from an ordinary `GET /orders/:id`, or from the cache that checkout filled a moment earlier.

The order list and the kitchen board do not use a stream. They refetch every five seconds, which suits a list of many orders better than one connection for each.

What it costs

The stream polls the database, because the database is the only thing every instance can see. Each customer watching an order holds one function invocation open and causes one small read every two seconds. For a handful of concurrent orders that is fine.

At scale the polling would be replaced, not the design. Everything the stream knows about the outside world is the `loadOrder` function it is given. MongoDB change streams, or a message broker such as Redis pub/sub, could wake the stream on a write instead of on a timer, without touching the browser or the wire format.

2.11 Money and pricing

- **Amounts are integers in paise,** in the database, in the API and in every calculation. Floating-point rupees would drift: `0.1 + 0.2` is not `0.3`. The only place that converts to rupees is `formatPrice`, for display.
- **One function decides what an order costs.**

```
// src/shared/pricing.ts
export const DELIVERY_FEE = 4000;
export const FREE_DELIVERY_THRESHOLD = 50000;
export const TAX_PERCENT = 5;

export function calculateTotals(lines: readonly PricedLine[]): OrderTotals {
  const subtotal = lines.reduce(
    (sum, line) => sum + line.unitPrice * line.quantity,
    0,
  );

  if (subtotal === 0) {
    return { subtotal: 0, deliveryFee: 0, tax: 0, total: 0 };
  }

  const deliveryFee = subtotal >= FREE_DELIVERY_THRESHOLD ? 0 : DELIVERY_FEE;
  const tax = Math.round((subtotal * TAX_PERCENT) / 100);

  return { subtotal, deliveryFee, tax, total: subtotal + deliveryFee + tax };
}
```

Rule	Value
Delivery fee	₹40
Free delivery	When the subtotal is ₹500 or more
Tax	5% of the subtotal, rounded to the nearest paisa
Empty cart	Everything is zero. No delivery fee is charged on nothing.

- **The browser uses it to preview the bill. The server uses it to decide the bill.** The server feeds it prices read from the database, so the result does not depend on anything the browser sent. A test places an order while sending false prices, totals and a status, and checks that all of them are ignored.
- **The price paid is kept.** An order stores each dish's name and unit price as they were when it was placed. A test changes the menu price afterwards and checks that the order still shows what was charged.
- **Display.** `formatPrice` uses the `en-IN` locale: `₹249` for a whole number of rupees, `₹22.35` otherwise, with Indian digit grouping for large amounts.

2.12 Validation

Every rule about what a request may contain is a Zod schema in `src/shared/schemas.ts`.

Schema	Used by	Rules
`deliveryDetailsSchema`	The checkout form, and inside `createOrderSchema`	`name`: trimmed, 2 to 80 characters. `address`: trimmed, 10 to 300. `phone`: spaces, dashes and brackets removed, then an optional + and 10 to 15 digits.
`orderItemSchema`	Inside `createOrderSchema`	`menuItemId`: a 24-character hexadecimal id. `quantity`: a whole number from 1 to 20.
`createOrderSchema`	`POST /orders`	`items`: 1 to 30 entries, with no dish listed twice. `customer`: the delivery details.
`updateOrderStatusSchema`	`PATCH /orders/:id/status`	`status`: one of the five statuses.
`listOrdersQuerySchema`	`GET /orders`	`status`: optional, one of the five statuses.
`registerSchema`	The sign-up form and `POST /auth/register`	`name`: trimmed, 2 to 80. `email`: trimmed, lower-cased, a valid address. `password`: 8 to 72 characters.
`loginSchema`	The sign-in form and `POST /auth/login`	`email`: as above. `password`: 1 to 72 characters.

How the rules are applied:

- **The API never trusts that the form ran.** Every request body is parsed again on the server with the same schema. Anyone can call the API without the form.
- **The form and the API cannot disagree,** because they import the same schema object. Changing a rule or its message changes both.
- **Validation also cleans.** The schemas trim, lower-case emails and strip the punctuation from phone numbers, and the handler receives the cleaned values. `+91 98765 43210` is stored as `+919876543210`.
- **Unknown fields are dropped.** A `role` sent to sign-up, or a `total` sent with an order, never reaches the handler.
- **Types are checked, not coerced.** A quantity sent as the string `"2"` is refused.
- **All problems are reported together,** keyed by field path, so a form can show every message beside its field in one round trip.
- **Passwords stop at 72 characters** because bcrypt reads at most 72 bytes. A longer password is refused rather than silently cut short.

In the browser, `zod-form-resolver.ts` connects react-hook-form to these schemas. It is a 30-line function written for this project. On success the form receives the parsed, cleaned values. On failure it receives the first message for each field, along with a reference to the field so that focus can move to the first one that needs fixing.

2.13 The menu catalogue and seeding

One source for the menu

The menu is defined in code, in `menu.catalog.ts`: an array of 43 entries, each with a slug, name, description, price in paise, photo path, section, vegetarian flag and position. `npm run seed` writes it into the database.

Editing the menu through the site was not part of the brief. Keeping the catalogue in code means the menu is versioned with the app, reviewed like any other change, and checked by tests.

A dish can never be on the menu twice

Three independent guards:

1. **The seed refuses a bad catalogue before it writes anything.** `findCatalogProblems` reports every slug, name or photo used by more than one dish. Names that differ only in capitals or spacing count as the same dish.
2. **Each dish is matched on its slug and updated in place** (an upsert). Running the seed again adds nothing. A dish keeps its database id, so orders that refer to it still do.
3. **MongoDB enforces a unique index on `slug`.**

Tests cover each guard. They also run against the real catalogue and check that every dish has its photo on disk, that no picture is stored under two file names, that every section has at least one dish, that no two dishes in a section share a position, and that every price is a positive whole number of paise.

The same idea continues further along: adding a dish to the cart twice raises its quantity instead of adding a second line, and the API rejects an order that lists a dish twice.

What `npm run seed` does

1. Connects to the database named by `MONGODB_URI` in `.env.local`.
2. Creates every index on all five collections (`syncIndexes`), rather than leaving that to the first request that needs one.
3. Makes the menu match the catalogue: upserts every dish and marks it available, then marks as unavailable any dish in the database that is no longer in the catalogue. Dishes are never deleted, because past orders still point at them.
4. Creates or updates the two demo accounts.
5. Prints what it did, for example `Menu: 43 dishes available, 0 taken off`.

It is safe to run any number of times, and it never touches orders.

The script runs with `tsx --conditions=react-server`. Modules marked `server-only` throw when loaded outside a server environment. That flag tells Node to resolve the marker the way a React server would, so the script can import the same models and services as the app.

Demo accounts

Account	Email	Password	Role
Demo customer	`asha@atithya.example`	`demo-customer-1`	`customer`
Kitchen staff	`kitchen@atithya.example`	`demo-kitchen-1`	`admin`

They are public on purpose. They are defined in `src/shared/demo-accounts.ts`, and the sign-in page offers them as one-click buttons so that a reviewer can try both roles without signing up. A real deployment would not have them.

Changing the menu

- **A dish, price, description or photo:** edit the entry in `menu.catalog.ts`. A new photo goes in `public/menu/`. Then run the seed.

- **Removing a dish:** delete its entry. The seed takes it off the menu and leaves it in the database.
- **A new section:** add it to `MENU_CATEGORIES` and `CATEGORY_LABELS` in `src/shared/types.ts`. The menu page, its filter bar and the database's validation all follow from that one list.

When the change is headed for the live site, deploy first and seed second. Section 2.20 explains why.

2.14 Security

Risk	Defence	Where
A tampered request changes what an order costs	The request carries ids and quantities only. Prices are read from the database and the bill is computed on the server. Extra fields are dropped.	`order.service.ts`, `pricing.ts`, `schemas.ts`
Seeing or cancelling another customer's order	Every read checks ownership. Someone else's order answers `404`, not `403`, so ids cannot be probed.	`loadVisibleOrder` in `order.service.ts`
A customer acting as staff	Staff endpoints declare `auth: "admin"`. The role is inside the signed token. Sign-up never reads a role from the request.	`route.ts`, `auth.service.ts`
A script in the page stealing the session	Both tokens are in httpOnly cookies, which script cannot read. React escapes everything it renders, and the code never injects raw HTML.	`cookies.ts`
Another website acting with the user's cookies (CSRF)	State-changing requests whose `Origin` is another host are refused, on top of `SameSite=Lax` cookies.	`assertSameOrigin` in `route.ts`
A stolen refresh token	Each refresh token works once. A used one coming back ends the whole session. Only a hash of the token's id is stored.	`refreshSession` in `auth.service.ts`
Password guessing	10 sign-in or sign-up attempts a minute per address. Passwords are hashed with bcrypt.	`rate-limit.ts`, `password.ts`
Finding out which emails have accounts by signing in	A wrong password and an unknown email give the same answer after the same amount of work.	`loginUser` in `auth.service.ts`
Flooding the kitchen with orders	10 orders a minute per address, at most 30 lines in an order and 20 of a dish.	`rate-limit.ts`, `schemas.ts`
Query injection	Every value is validated as the right type before it reaches a query. Ids must be 24 hexadecimal characters, so an object can never stand in for one.	`schemas.ts`, `order.repository.ts`
A sign-in link that redirects to another website	The `?next=` destination is accepted only if it is a path inside this site.	`safe-next-path.ts`
Clickjacking and content sniffing	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, a strict `Referrer-Policy`, and a `Permissions-Policy` that turns off camera, microphone and location.	`next.config.ts`
Leaking internals in errors	Unexpected errors return a generic 500, and the cause goes only to the server log. The `X-Powered-By` header is switched off.	`respond.ts`, `next.config.ts`
Weak or leaked secrets	The app refuses to start with a signing secret under 32 characters or with two identical secrets. `.env` files are ignored by git.	`env.ts`, `.gitignore`
Server code reaching the browser	Modules holding configuration and the database connection are marked `server-only`.	`env.ts`, `connect.ts`

How the rate limit works

Serverless instances share no memory, so a counter held in a variable would be a separate counter for each instance. The count lives in MongoDB instead, where every instance sees it:

```
// src/server/http/rate-limit.ts
const counter = await RateLimitModel.findOneAndUpdate(
  { key: `${rule.name}:${clientAddress(request)}:${windowStart}` },
  { $inc: { count: 1 }, $setOnInsert: { expiresAt: new Date(windowEnd) } },
  { upsert: true, returnDocument: "after" },
).lean();

if (counter.count > rule.limit) {
  throw errors.rateLimited(Math.ceil((windowEnd - now) / 1000));
}
```

It is a fixed-window counter. The key names the rule, the client's address and the start of the current minute. One atomic upsert both creates the counter and increments it, so concurrent requests cannot miscount. A TTL index deletes each counter when its window ends. The client's address is the first entry of the `X-Forwarded-For` header, which the hosting platform's proxy sets.

There are two rules, each allowing 10 requests a minute per address: `auth`, shared by sign-in and sign-up, and `orders`, for placing an order. They are counted separately, so failed sign-ins do not use up the order allowance.

Where the protection stops

- Sign-up says when an email is already registered. It has to tell the person something. This means sign-up, unlike sign-in, can be used to test whether an email has an account, and only the rate limit slows that down.
- There is no Content-Security-Policy header yet. A strict one needs a fresh nonce for every request.
- Requests that carry no `Origin` header pass the origin check. Those come from tools such as `curl`, which have no victim's cookies to abuse.
- The kitchen's demo account is public, so on the demo anyone can move or cancel any order. That is deliberate, for review.

2.15 Front end

How pages are rendered

The app uses Next.js Cache Components (`cacheComponents: true`), which lets one page mix static and per-request content. The production build reports:

Route	Rendering	What that means
`/`	Partial prerender	The banner and page frame are static HTML from the CDN. The menu is read from the database for each request and streams in under a `<Suspense>` boundary.
`/orders/[id]`	Partial prerender	A static shell. The order id is read per request, and the order itself is fetched by the browser.
`/checkout`, `/login`, `/register`, `/orders`, `/admin/orders`	Static	Shells served from the CDN. Their data is fetched by the browser through the API.
`/api/v1/*`	Dynamic	Run for every request.

Three consequences:

- **The first paint is fast.** The menu page's shell does not wait for the database.
- **The build needs no database.** The one server-side read sits behind `await connection()`, which marks it as request-time, and the environment is read lazily. `next build` runs in CI with no secrets at all.
- **Private pages hold no private data.** A page such as `/orders` is the same static file for everyone. What differs is the data the browser then fetches with the visitor's own cookies, and the API decides what that is.

`partialPrefetching: true` makes a link prefetch the destination's static shell, so moving between pages feels immediate.

State

What	Where it lives	Notes
The signed-in user	TanStack Query, key `["auth", "me"]`	Treated as fresh for 60 seconds. Shared by the header and every page.
Order lists	TanStack Query, key `["orders", "list", <status or "any">]`	Refetched every 5 seconds while it matters
A single order	TanStack Query, key `["orders", "detail", <id>]`	Written by fetches, by mutations, and by pushed events
The cart	A Zustand store saved to `localStorage` under `atithya-cart`	Exists only in the browser until checkout
Whether the cart panel is open	A second, tiny Zustand store	So that any button on any page can open it
Form fields	react-hook-form	Local to each form
The chosen menu section	`useState` in `MenuBrowser`	Local to the menu

Server data and client state are kept apart on purpose. TanStack Query owns everything that comes from the API, and the cart is the only real client state.

A few details of how the cache is used:

- A `4xx` answer is never retried, because asking again will not change it. Network failures and `5xx` answers are retried twice.
- When checkout succeeds, the new order is written straight into the cache, so the tracking page renders at once instead of showing a loading state.
- When the signed-in person changes, the cached user is replaced and every cached order is thrown away, so one person's orders can never be shown to the next person on the same browser.
- After a cancellation only the order lists are marked stale. The cancelled order itself is already current, and refetching it would be a wasted request.

The cart and hydration

The server knows nothing about the cart, so it always renders an empty one. If the store read `localStorage` as it was created, the browser's first render would differ from the server's HTML and React would report a mismatch. So:

- The store is created with `skipHydration`, and starts empty in the browser as well.
- `CartHydration`, mounted once in the root providers, loads the saved cart after the page has mounted.
- `CartGate` holds back content that would be wrong before the saved cart is known. The checkout page uses it so that it never flashes "Your cart is empty" at someone whose cart is about to load.

The store caps each dish at 20 and the cart at 30 different dishes, the same limits the API's schema uses.

The API client

`src/lib/api-client.ts` is the only place the browser calls `fetch` for the API. It unwraps `{ data }`, turns error responses into an `ApiError` carrying the status, code, message and field details, and handles session refresh as described in 2.7.

It adds two codes of its own for failures that never reached the server's error format: `NETWORK_ERROR`, when the server could not be reached, and `UNEXPECTED_RESPONSE`, when the reply was not JSON. Both come with a message fit to show a customer.

Components

Primitives in `src/components/ui/` know nothing about food or orders:

Component	What it is
`Button`, `buttonStyles`	Five variants, three sizes, a full-width option and a loading state. `buttonStyles` gives a link the same look.
`TextField`, `TextAreaField`	A label, control, hint and error, wired together for assistive technology
`Sheet`	A panel that slides in from the right, built on Radix Dialog
`QuantityStepper`	Minus, quantity, plus, with spoken labels that name the dish
`StatusBadge`	An order status as a coloured label
`VegMark`	The vegetarian or non-vegetarian mark
`Notice`	The box used for every empty state and failure: a title, an explanation and a next step
`Skeleton`	A placeholder block shown while content loads

Features in `src/features/` each own one part of the product:

Feature	Contents
`menu`	`MenuBrowser` (sections and filtering), `SectionStrip` (the scrolling filter bar), `MenuItemCard`
`cart`	The store, `CartPanel`, `CartSheet` with the header button and the phone's bottom bar, `PriceBreakdown`, hydration
`checkout`	`CheckoutForm` (the fields) and `CheckoutView` (the sign-in gate, the summary, placing the order)
`orders`	`order-api.ts`, `useOrderStream`, `OrderTrackingView`, `OrderStatusTracker`, `OrderSummary`, `OrdersListView`
`auth`	`useCurrentUser`, the sign-in, sign-up and sign-out mutations, `AuthForm`, `AuthPage`
`admin`	`AdminOrdersView`, the kitchen board

Pages in `src/app/` only arrange these. A page file is typically under 20 lines.

Design system

The palette is taken from the Atithya logo: its deep green for the brand, its gold for the one accent. The colours are declared once, as Tailwind theme tokens in `src/app/globals.css`, and components use the token names (`bg-leaf-900`, `text-muted`, `border-line`), never raw colour values.

Token	Value	Used for
`leaf-900`	`#043022`	The brand green: the banner, primary buttons, the chosen filter
`leaf-950`, `leaf-800`, `leaf-700`	`#02241a`, `#0a4331`, `#136046`	Pressed and hover states, the focus outline, the veg mark
`leaf-200`, `leaf-100`, `leaf-50`	`#c3dccd`, `#ddebe2`, `#eff5f1`	Tints: hover backgrounds, placeholders
`gold-500`	`#c19852`	The accent: the cart badge, the plate's rim
`gold-600`, `gold-300`, `gold-100`	`#a67f3c`, `#dfc898`, `#f6eedd`	The current-stage dot, text selection, the free-delivery hint
`chili-700`, `chili-600`, `chili-50`	`#a3271a`, `#c4321e`, `#fbece9`	Errors, cancellation, the non-veg mark

Token	Value	Used for
`ink`, `muted`	`#13231c`, `#55665d`	Body text and secondary text
`line`, `mist`, `paper`	`#dce4de`, `#f4f7f4`, `#ffffff`	Borders, panel backgrounds, the page

Two typefaces, loaded through `next/font`:

- **Anek Latin** sets all interface text. It is drawn by an Indian type foundry and carries the rupee sign well.
- **Young Serif** is used only for headings.

Dish photos are stored in the repository under `public/menu/` and served through `next/image`, which resizes them for the size they are shown at. The photos are from Unsplash.

Accessibility in the code

- A "Skip to content" link is the first focusable element on every page.
- One `:focus-visible` rule gives every control the same outline.
- `TextField` connects its label with `htmlFor`, marks a bad value with `aria-invalid`, points to its message with `aria-describedby`, and gives the message `role="alert"` so it is announced.
- The quantity is in an `aria-live` region. The filter buttons use `aria-pressed`. The current stage on the timeline has `aria-current="step"`.
- The timeline writes "Completed", "In progress" or "Not started" for each stage in text that only screen readers receive.
- Icons are `aria-hidden`. A button that shows only an icon on small screens keeps its name in screen-reader text.
- Decorative images have an empty `alt`. Dish photos are described by the dish's name.
- The filter bar's arrows are a convenience for the mouse. They are kept out of the tab order and hidden from screen readers, because every section button is already reachable by keyboard.
- A `prefers-reduced-motion` rule in CSS stops animations and transitions, and the scrolling code checks the same preference before scrolling smoothly.

2.16 One order from start to finish

This follows a single order through every layer, naming the code involved at each step.

1. **The menu page loads.** The CDN serves the static shell from `src/app/page.tsx`: the banner and a skeleton. On the server, the `Menu` component awaits `connection()`, calls `listMenu()`, and streams `MenuBrowser` with the 43 dishes into the page.
2. **A dish is added.** `MenuItemCard` calls `add(item)` on the cart store. The store appends a line, or raises the quantity if the dish is already there, and saves to `localStorage`. The header badge and the phone's bottom bar re-render, because they select from the same store.
3. **Checkout opens.** `/checkout` is a static shell. `CartGate` waits for the saved cart. `useCurrentUser` asks `GET /auth/me`. A signed-in customer sees `CheckoutForm` with their name filled in.
4. **The form is submitted.** react-hook-form validates with `deliveryDetailsSchema`. If it passes, a mutation posts `{ items: [{ menuItemId, quantity }], customer }` to `/orders`. The cart's prices are not sent.
5. **The request passes through `route()`:** origin check, database connection, the order rate limit, the access token, then validation with `createOrderSchema`.
6. **`placeOrder` runs.** It loads the dishes, copies their names and prices from the database, prices the order with `calculateTotals`, and inserts it as `received` with `nextTransitionAt` 20 seconds ahead. The API answers `201` with the order.

7. **The browser moves on.** The mutation writes the order into the query cache, marks the order lists stale, empties the cart and navigates to `/orders/<id>`.
8. **The tracking page renders at once** from the cached order: the headline "We have your order", the arrival time, the timeline, and the cancel button. `useOrderStream` opens an `EventSource` on `/orders/<id>/events`.
9. **The stream starts.** The events route checks that the caller may see the order, then returns the stream. Its first poll sends the order, and the indicator reads "Live updates on".
10. **Twenty seconds pass.** Nothing has been running. On the stream's next poll, `getOrder` calls `loadVisibleOrder`, which sees that the order is due and calls `settleDueOrders`. The order becomes `preparing`, stamped with the time it was due. Its version has changed, so the stream sends it.
11. **The page updates.** The hook writes the pushed order into the cache. The headline becomes "Your food is being prepared", the timeline moves on, and the cancel button disappears, because `canCustomerCancel("preparing")` is false.
12. **Staff step in.** The kitchen board has been refetching `GET /orders` every five seconds. A member of staff presses "Send out for delivery". `PATCH /orders/<id>/status` checks `canTransition`, then writes the change only if the order is still `preparing`. The countdown to the next stage restarts.
13. **The customer sees it within two seconds,** on the stream's next poll. The arrival time moves earlier.
14. **The order is delivered,** by the simulation or by staff. The stream sends the final order and closes. The page shows "Delivered. Enjoy your meal.", the live indicator disappears, and "Order something else" appears.

If the customer's access token expires along the way, the next API call gets `401`, the client refreshes the session once and repeats the call. If the stream is refused for the same reason, the hook refreshes the session and reconnects.

2.17 Testing

Approach

The project was written test-first. For the business rules, the API and the key components, the git history shows a commit of failing tests, marked `(red)`, ahead of the commit that makes them pass. There are 18 such red commits, for example:

```
29a4739 test(orders): place, read, list, update status and cancel API (red)
bf22691 feat(orders): place, read, list, update status and cancel orders
64cf568 test(orders): time-driven status simulation and delivery estimate (red)
64479dd feat(orders): simulate kitchen progress by settling overdue steps on read
```

Page composition and styling primitives have no logic of their own. They were checked by the production build and by driving the app in a real browser.

Two rules were followed throughout:

- **Test behaviour through the real code.** The API tests call the real route handlers against a real MongoDB. The UI tests render the real components. Only the true edges are replaced: `fetch` and `EventSource` in the browser tests, and the clock.
- **Never wait for time.** Token expiry, the kitchen simulation, rate-limit windows and the stream's polling are all tested by moving a fake clock.

Three test projects

`vitest.config.mts` defines three projects that run together under `npm test`.

Project	Tests	Environment	What it covers
`unit`	94	Node	Pure logic in `src/shared/` and `src/server/`: no database, no DOM

Project	Tests	Environment	What it covers
`api`	109	Node	Every endpoint, through the real route handlers, against an in-memory MongoDB
`ui`	136	jsdom	Components, hooks and browser helpers in `src/features/`, `src/components/` and `src/lib/`

Unit tests (94)

File	Tests	Covers
`src/shared/schemas.test.ts`	29	Every validation rule: names, addresses, phone formats, quantities, duplicate dishes, emails, passwords
`src/shared/order-status.test.ts`	15	The next status, legal and illegal moves, who may cancel, which statuses are final
`src/server/modules/menu/menu.catalog.test.ts`	13	Duplicate detection, and the real catalogue: photos on disk, no repeats, valid prices
`src/server/modules/auth/tokens.test.ts`	11	Token contents, expiry at 15 minutes and 7 days, wrong secret, wrong token type, malformed tokens
`src/server/modules/orders/order-events.test.ts`	10	The stream: first event, change detection, 2-second polling, keep-alive, closing
`src/shared/pricing.test.ts`	6	Totals, the delivery fee on either side of the ₹500 threshold, tax rounding, the empty cart
`src/shared/money.test.ts`	5	Rupee formatting
`src/server/config/env.test.ts`	5	Environment validation: missing values, short secrets, identical secrets

API tests (109)

File	Tests	Covers
`tests/api/orders.test.ts`	42	Create, read, list, update status and cancel, with the permission and validation failures of each
`tests/api/auth.test.ts`	24	Sign-up, sign-in, `/auth/me`, refresh rotation, reuse detection, sign-out, cross-site refusal
`tests/api/order-simulation.test.ts`	16	Each step at its due time, catching up, exactly-once under concurrency, the simulation alongside manual changes, the estimate
`tests/api/menu-seed.test.ts`	9	Seeding twice adds nothing, updates in place, retiring and restoring dishes, refusing a duplicated catalogue
`tests/api/menu.test.ts`	6	The menu listing, its ordering, unavailable dishes, the empty menu
`tests/api/order-events.test.ts`	5	The stream's headers, a finished order, a pushed change, another customer's order, signing in
`tests/api/rate-limit.test.ts`	5	The 11th attempt, `Retry-After`, separate addresses, proxies, the window resetting
`tests/api/route-wrapper.test.ts`	2	A generic 500 for an unexpected failure, and framework errors passed through

UI tests (136)

File	Tests	Covers
`src/features/cart/cart-store.test.ts`	19	Add, increase, decrease, remove, at most 20 of a dish and 30 different dishes, totals, surviving a reload

File	Tests	Covers
src/features/orders/order-tracking-view.test.tsx	17	The status display, live pushes, reconnecting, session renewal, cancelling, missing orders
src/features/menu/menu-browser.test.tsx	16	Sections, filtering, keeping the menu in view, the scrolling filter bar and its arrows
src/lib/api-client.test.ts	13	Success and error handling, and every case of session refresh
src/features/menu/menu-item-card.test.tsx	11	What a dish shows, veg marks, adding, the quantity control, the cap of 20, a full cart
src/features/cart/cart-panel.test.tsx	10	The empty cart, lines, the bill, the free-delivery hint, removing dishes
src/lib/safe-next-path.test.ts	10	In-app paths kept, and every form of external address refused
src/features/auth/auth-form.test.tsx	8	Signing in, signing up, the demo buttons, validation before any request
src/features/checkout/checkout-form.test.tsx	8	Field validation, server errors shown beside fields, no double submission
src/features/checkout/checkout-view.test.tsx	7	Placing an order, emptying the cart, refused orders, the sign-in gate, the empty cart
src/features/admin/admin-orders-view.test.tsx	7	The board, moving an order on, finished orders, refused changes, turning customers away
src/features/orders/order-status-tracker.test.tsx	7	The timeline in each status, including cancelled
src/features/orders/orders-list-view.test.tsx	3	The list, the empty state, the signed-out state

How the API tests work

- **A real database.** `mongodb-memory-server` starts one throwaway MongoDB for the run. Each test file gets its own database inside it, so files run in parallel without seeing each other's data. Collections are emptied before every test.
- **Real handlers.** `tests/helpers/http.ts` is a minimal stand-in for a browser. It calls the route handlers directly with a `NextRequest` and carries cookies between calls, honouring each cookie's path as a browser does. One client is one browser session, so two clients are two separate users. This is how the tests show that one customer cannot see another's order.
- **Realistic set-up.** A test user is created by calling the real sign-up endpoint, not by writing to the database. Each one comes from its own network address, so the rate limit never interferes with a test about something else.
- **A fake clock.** `vi.useFakeTimers({ toFake: ["Date"] })` replaces only `Date`. The tests can jump 20 seconds ahead while the database driver's own timers keep working.

How the UI tests work

- Components are rendered with React Testing Library and driven as a user would drive them: by role, label and visible text.
- `mockApi` replaces `fetch` with a table of canned replies keyed by method and path. A request the test did not expect fails loudly.
- jsdom has no `EventSource`, so `FakeEventSource` stands in for it. A test opens the connection, pushes an order, drops the connection, or refuses it, and checks what the page does.
- jsdom has no layout engine either. The set-up file supplies harmless stand-ins for scrolling and `ResizeObserver`, which individual tests can spy on.

Running the tests

Command	What it does
`npm test`	Everything, once
`npm run test:watch`	Watch mode
`npx vitest run --project api`	One project: `unit`, `api` or `ui`
`npx vitest run tests/api/orders.test.ts`	One file

The tests need no database of your own and no `.env` file. The first run downloads a MongoDB binary for the in-memory server and caches it.

2.18 Configuration

Configuration comes from environment variables, validated by a Zod schema in `src/server/config/env.ts`.

Name	Required	Purpose
`MONGODB_URI`	Yes	The MongoDB connection string
`JWT_ACCESS_SECRET`	Yes	Signs access tokens. At least 32 characters.
`JWT_REFRESH_SECRET`	Yes	Signs refresh tokens. At least 32 characters, and different from the access secret.
`ORDER_STEP_SECONDS`	No	Seconds a simulated order spends in each status. A whole number from 1 to 3600. Defaults to 20.
`SITE_URL`	No	The site's public address, used for link previews. Without it, the domain Vercel reports at build time is used.
`NODE_ENV`	No	Set by the tooling. In `production`, cookies are marked `Secure`.

- **A bad configuration fails loudly.** The error lists every problem by variable name, for example `JWT_REFRESH_SECRET: must be different from JWT_ACCESS_SECRET`.
- **The environment is read on first use, not at import.** That is what lets `next build` load the server modules without the runtime secrets being present.
- **`.env.example`** lists every variable with a comment. Real values go in `.env.local`, which git ignores.

A secret can be generated with:

```
node -e "console.log(require('crypto').randomBytes(36).toString('base64url'))"
```

2.19 Running it locally

You need Node.js 22.12 or newer and a MongoDB connection string. A free MongoDB Atlas cluster works, as does a local server.

```
npm install
cp .env.example .env.local   # then fill in the three required values
npm run seed                 # loads the menu and the two demo accounts
npm run dev                  # http://localhost:3000
```

In PowerShell, the second line is `Copy-Item .env.example .env.local`.

Script	What it does
`npm run dev`	The development server
`npm run build`	A production build. Needs no database.
`npm start`	Serves the production build
`npm test`	All tests, once. Needs no database and no `.env` file.
`npm run test:watch`	Tests in watch mode
`npm run lint`	ESLint
`npm run typecheck`	Generates Next.js route types, then runs the TypeScript compiler
`npm run format`	Formats every file with Prettier
`npm run format:check`	Checks formatting without changing anything
`npm run seed`	Loads the menu and the demo accounts. Safe to run again.

To watch an order move faster while developing, set `ORDER_STEP_SECONDS` to a smaller number in `.env.local`.

A note on dependencies. The lockfile was generated with npm 10, the version that ships with Node 22 and that CI uses, and it is also accepted by npm 11. After adding or upgrading a package, run `npm ci` and `npm test` from a clean checkout. npm has been seen to drop the platform-specific optional packages that the test runner's bundler needs when a single package is installed into an existing tree, and the symptom is a "Cannot find native binding" error from Vitest. Removing `node_modules` and the lockfile and installing again fixes it.

2.20 Deployment and operations

Hosting

The app is hosted on Vercel, connected to the GitHub repository. Every push to `main` is built and deployed to production automatically.

- **Region.** `vercel.json` pins the server code to Vercel's Mumbai region (`bom1`), because the MongoDB Atlas cluster is in India. Vercel's default region is in the United States, which would add an intercontinental round trip to every database query.
- **Environment variables on Vercel:** `MONGODB_URI`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` and `SITE_URL`. `ORDER_STEP_SECONDS` is not set, so the live site uses the default of 20 seconds.
- **Database access.** Atlas has to accept connections from anywhere (`0.0.0.0/0`), because Vercel's outgoing addresses are not fixed. Access is controlled by the database user's credentials.
- **The live stream.** The events route declares `maxDuration = 300` seconds, and the stream ends itself at 240.
- **The connection pool.** The MongoDB connection is cached on `globalThis`, so a warm function instance reuses one pool across requests instead of opening a connection each time. A failed attempt is not cached, so the next request tries again.

Continuous integration

`.github/workflows/ci.yml` runs on every push to `main` and on every pull request, on Node 22:

1. `npm ci`
2. `npm run format:check`
3. `npm run lint`

4. `npm run typecheck`
5. `npm test`
6. `npm run build`

The MongoDB binary used by the API tests is cached between runs. The build step proves that the app builds with no database and no secrets.

Routine tasks

Changing the menu on the live site. Do it in this order:

1. Commit and push the change to `menu.catalog.ts`, along with any new photos.
2. Wait for the Vercel deployment to finish.
3. Run `npm run seed` against the production database.

The seed writes to whichever database `MONGODB_URI` in `.env.local` names. Seeding first would make the live menu refer to photos or sections that the deployed code does not have yet.

Changing how fast orders move. Set `ORDER_STEP_SECONDS` in Vercel and redeploy. Orders already in progress keep the due time they were given. The new length applies from each order's next step.

Changing the database password. Update `MONGODB_URI` in Vercel and in `.env.local`, then redeploy.

Changing the JWT secrets. Update them in Vercel and redeploy. Every existing token stops verifying, so everyone is signed out and signs in again.

Finding out what went wrong. Unexpected errors are written to the function's log as `Unhandled error in route handler`, followed by the error. They appear in Vercel's logs for the deployment. The client is never shown the cause.

2.21 Design decisions and trade-offs

One Next.js app for pages and API, instead of a separate API server. The app has to deploy on Vercel, and one deployable is simpler to run than two. It also lets the browser and server share TypeScript types and validation schemas directly, and there is no cross-origin configuration because both live on one origin. The cost is that the API is tied to Next.js route handlers. The business logic is not: it sits in plain functions under `src/server/modules/` that know nothing about HTTP.

MongoDB, instead of keeping orders in memory. The brief allows in-memory storage. On a serverless host that would not work: instances share no memory and are recycled at will, so an order placed through one instance would be invisible to the next. A database is the only thing all instances can see.

Server-Sent Events, instead of WebSockets. Status tracking is one-way. SSE is plain HTTP, runs in a serverless function, needs no extra service, and reconnects by itself. Sockets would need either a server that stays up or a hosted realtime service, and the second would break the rule that MongoDB is the only external service. The cost is that the stream polls the database, discussed in 2.10.

Settling the simulation on read, instead of timers or a scheduled job. No process stays up to hold a timer, and a scheduled job is far coarser than a 20-second step. Storing when each step is due and applying overdue steps on read gives the same result with nothing running. The cost is that an order nobody reads does not change in the database until someone does read it. That is invisible to users, because every read settles first and every change is stamped with the time it was due.

Tokens in httpOnly cookies, instead of `localStorage`. Script cannot read an httpOnly cookie, so a scripting bug cannot leak the session. `EventSource` also cannot send custom headers, but does send cookies. The cost is that cookies bring CSRF into play, answered with the origin check and `SameSite=Lax`.

Refresh tokens backed by a session record, instead of tokens alone. A token alone cannot be revoked. The session record is what makes sign-out real and lets a stolen refresh token be detected. The cost is one database write for each refresh, roughly every 15 minutes for an active user.

Rate-limit counters in MongoDB, instead of Redis or process memory. Process memory is not shared between instances, and Redis is another service. One atomic upsert in MongoDB is correct under concurrency, and a TTL index cleans up. The cost is a database write on every rate-limited request, which is why only three endpoints are limited.

Cancelling instead of deleting. `DELETE /orders/:id` sets the status to `cancelled` and never removes the document. Customers keep their history, staff keep a record of what was cancelled, and nothing else in the system has to cope with an order that vanished.

`404` instead of `403` for someone else's order. A `403` would confirm that the id belongs to a real order. Answering "not found" gives away nothing.

The menu catalogue in code, instead of an editing interface. Menu editing was not in the brief. A catalogue file is versioned, reviewed and tested, and the duplicate checks in 2.13 run before anything reaches the database. The cost is that changing the menu needs a developer.

A small in-house form resolver, instead of `@hookform/resolvers`. That package's optional peer dependencies made the lockfile differ between npm 10 and npm 11, and CI rejected it. Thirty lines of code removed the dependency and the problem.

TanStack Query and Zustand, instead of one global store. Nearly all the app's state is server data, which a query cache handles better than hand-written reducers: caching, refetching, staleness and error states come built in. The cart is the only real client state, and a 100-line store is enough for it.

2.22 Known limitations and next steps

- **Fixed-window rate limiting** allows a short burst across a window boundary: 10 requests at the end of one minute and 10 at the start of the next. A sliding window would be stricter.
- **The rate limit is per network address.** People behind one address, such as an office, share an allowance. In local development, where there is no proxy, every request shares one bucket.
- **A simultaneous refresh from two tabs can end the session,** because each refresh token works once and the second tab's token looks like a replay. A short grace period for the previous token would smooth that over.
- **An access token cannot be revoked** before its 15 minutes run out.
- **No Content-Security-Policy yet.** A strict one needs a nonce for every request.
- **The live stream polls the database** every two seconds for each customer watching. At scale, change streams or a message broker would replace the polling behind the same `loadOrder` seam.
- **Order lists return the 50 most recent orders,** with no pagination.
- **The cart is kept per browser, not per account.**
- **The staff demo account is public,** so anyone can move or cancel any order on the demo. That is deliberate for review and would not exist in production.
- **No automated browser tests.** The full flow was checked by hand in a real browser. A small end-to-end suite would guard it against regressions.
- **Not built:** payments, editing the menu from the site, multiple restaurants, notifications, saved addresses, password reset and email verification.

2.23 How it was built

The project was built by pairing with Claude Code, Anthropic's coding agent, directed step by step.

- **Requirements and design first.** The brief was broken into a design before any code was written: the stack, the hosting constraints, the shape of the API and the security model. The serverless real-time design came from checking Vercel's current documentation rather than assuming.
- **Test-driven implementation.** Each behaviour began as a failing test, followed by the smallest implementation that passed it. The red and green commits are in the history.
- **Debugging.** A production build revealed that the API wrapper was swallowing one of Next.js's control-flow errors. A test that passed or failed depending on the wall clock was found and made deterministic. A lockfile that one version of npm accepted and another rejected was traced to a single dependency and removed.
- **Verification.** Every change went through the same checks CI runs: formatting, lint, type-check, the full test suite and a production build. The finished app was then driven in a real browser, at desktop and phone widths: menu, cart, checkout, live tracking and the kitchen board.

2.24 Where to find things

To change or understand	Look in
The delivery fee, the free-delivery threshold, tax	`src/shared/pricing.ts`
The statuses and which moves are legal	`src/shared/order-status.ts`
A validation rule or its message	`src/shared/schemas.ts`
A dish, a price, a description, a photo	`src/server/modules/menu/menu.catalog.ts`, then `npm run seed`
The menu's sections and their order	`src/shared/types.ts`
How every endpoint is guarded	`src/server/http/route.ts`
Error codes and messages	`src/server/http/errors.ts`
The rate limits	`src/server/http/rate-limit.ts`
Cookie names and flags	`src/server/http/cookies.ts`
Token lifetimes and contents	`src/server/modules/auth/tokens.ts`
Sign-in, sign-up, refresh and sign-out	`src/server/modules/auth/auth.service.ts`
Order rules: who may see, cancel or move an order	`src/server/modules/orders/order.service.ts`
Order queries, and the atomic updates	`src/server/modules/orders/order.repository.ts`
The simulated kitchen's timing	`src/server/modules/orders/order-simulation.ts`, and `ORDER_STEP_SECONDS`
The live stream: polling, keep-alive, limits	`src/server/modules/orders/order-events.ts`
The live stream in the browser	`src/features/orders/use-order-stream.ts`
How the browser calls the API and refreshes sessions	`src/lib/api-client.ts`
The cart	`src/features/cart/cart-store.ts`
Colours and fonts	`src/app/globals.css`, `src/app/layout.tsx`
Security headers and rendering settings	`next.config.ts`
The hosting region	`vercel.json`
The demo accounts	`src/shared/demo-accounts.ts`
Environment variables	`src/server/config/env.ts`, `.env.example`

2.25 Glossary

Term	Meaning here
Access token	A signed token, valid for 15 minutes, that proves who is making an API request
Refresh token	A signed token, valid for 7 days, used only to obtain a new access token
Rotation	Replacing the refresh token every time it is used, so that each one works once
Reuse detection	Ending a session when a refresh token that was already used is presented again
DTO	Data transfer object: the shape of a piece of data as it crosses the network, defined once in `src/shared/types.ts`
Paise	The hundredth part of a rupee. All money is stored and calculated as a whole number of paise.
Settle on read	Applying an order's overdue simulated steps at the moment the order is read, instead of on a timer
Server-Sent Events, SSE	A web standard in which the server keeps one HTTP response open and writes events to it as they happen
Static shell	The part of a page that is the same for everyone and is served from the CDN before any data arrives
Partial prerendering	Serving a static shell at once and streaming the per-request parts into it
Slug	A dish's stable text identifier, such as `margherita-pizza`, used to match it when seeding
Idempotent	Safe to run more than once with the same result. The seed script is idempotent.
Single-flight	Sharing one in-progress operation among every caller who needs it. The API client refreshes a session this way.
Terminal status	A status after which nothing more happens to an order: `delivered` or `cancelled`
TTL index	A MongoDB index that deletes a document by itself once a date stored in it has passed
Upsert	An update that inserts the document if it does not exist yet